

多模态大模型驱动的 Mobile-use Agent 记忆管理研究

RAVEN-M 的构建、因果诊断、假设检验与科学止损

项目：夏令营科研考核课题三

学号：2351441

姓名：许奕

2026 年 8 月 8 日

摘要

本文研究多模态大模型驱动的 Mobile-use Agent 应当如何保存和使用任务历史。核心困难并不只是上下文容量有限，而是“记忆被写入或引用”与“记忆对任务有净价值”之间存在一条容易被忽略的因果链：信息必须被正确观察、绑定到目标实体和字段，在需要时被检索，并且只对适当风险的动作具有授权力；随后动作还要被环境真实执行，结果与最终任务状态也必须被可靠验证。任何更靠前的接口、控制器或测量下限，都会使 memory efficacy 比较失去解释力。

为检验这一问题，本文在 AndroidWorld 上部署 Qwen3-VL-32B-Instruct，建立 B0-B3 四种历史基线，并实现包含 Planner、Executor、Memory Manager 与 Critic 的 RAVEN-M。系统完成 19 个 Hard tasks 上的 95 个五变体实验单元、两轮配对开发实验和连续案例诊断；其中 95-cell Hard breadth 记录 4,021 次模型调用和 17,971,842 tokens。RAVEN-M 的可审计性、跨 episode 隔离和 corruption invariants 均能工作，但完整方法 M0 未优于强简单基线 B3：Hard breadth 中二者均为 0/19，而 M0 增加约 33% 模型调用、53% tokens 和 52% wall time；r56 中 B3 为 4/4、M0 为 3/4，M0 的 prompt tokens 增加 69.53%。轨迹分析表明，主要瓶颈是动作接口、视觉定位、循环恢复、目标闭合和完成权限，而不是自然发生的跨月记忆过时。

基于负结果，本文曾把研究问题缩小到“正确事实是否会因为出现时机不当而绑定到错误目标”，并以 *Correct Memory, Wrong Target* 为候选机制开展三轮开发期因果筛查。实验冻结同一截图、事实、模型 revision、调用次数与 Early/Late 精确 token 匹配，共完成 96 个条件单元和 192 次模型调用。三轮 exact-value recall 均为 100%，但 Timing×Ambiguity 交互均为 0：候选目标信息过强时出现 grounding ceiling，去掉文字标签后出现 grounding floor，恢复屏幕原有标签后模型再次全部选对。结果不支持 reminder timing 假设，也不支持实现 Destination-First Binding Gate。

最后，本文回到 95-cell Hard breadth 的自然失败分布，筛出“系统已检测无进展并给出恢复反馈，下一步仍重复原动作”这一高频断点：80 个动作接口受支持的 cells

中，32 个至少连续重复同一动作 10 次；44 个可观察后续动作的 loop events 中，14 个立即重复原动作。最接近工作已覆盖相同触发条件、强制策略切换和闭环任务结局，因此本文停止继续为旧假设增加方法模块，但没有把“停止方法扩展”误写成“停止一切测量”。随后开展的四框架同条件复现先使 B3、M0、GUI-Owl-1.5 和 UI-Voyager 全部通过匹配 S1b 端到端资格门，再启动 4 任务、16 单元的 Hard pulse。服务器离线前完成 8 格，其中 7 格具有合法 evaluator 结果且均为 0 成功；H09 的 B3 轨迹因模型隧道中断被标为基础设施无效，不计作模型 0 分。部分轨迹进一步暴露了正确局部信息在计算变换、目标字段绑定和动作序列化处失效的现象，但样本尚不足以支持跨框架排名或新方法结论。

为排除自建控制器设置造成的能力下限，本文随后对齐 Qwen 官方公开的 Mobile Agent 运行方式，并在同一 Qwen3-VL-32B-Instruct revision 上完成 19 类 Hard tasks、3 个 seeds、共 57 个唯一任务单元。最终成功 7/57 (12.28%)，另有 2 个部分得分单元；完整运行包含 1,175 次可计入的模型调用。该结果证明此前 B3/M0 的全零不能等价解释为 Qwen 在 Hard 上完全无能力，同时也暴露 21 次错误完成声明、39 个重复状态 episode 和 14 个停滞 episode。基于此，我预注册了一次 matched L4 “动作后无页面变化则不承认动作语义已经实现”的最小干预。干预没有获得任务成功，且在 OsmAnd 中只是减少无变化循环后更早地错误结束。因果回放表明，更早的断点是把普通地点搜索误当成向轨迹加入途经点，即目标对象和操作角色在产生真实页面变化时就已绑定错误。这个负结果把下一步问题从笼统的“抑制重复动作”收缩为“在动作前验证当前对象是否具有完成该子目标的资格”。

关键词：Mobile-use Agent；多模态大模型；GUI Agent；记忆管理；AndroidWorld；证据绑定；可靠性

目录

1	引言	6
2	相关工作与研究切入点	7
2.1	端到端 Mobile GUI Agent	7
2.2	GUI Agent 的六类记忆机制	8
2.3	仅按“长短期”分类的不足	8
3	任务定义与评测原则	9
3.1	Agent 交互闭环	9
3.2	动作空间	9
3.3	公平性与证据等级	10
4	AndroidWorld Baseline	10
4.1	实验环境	10
4.2	端到端链路	11
4.3	历史基线	11
5	RAVEN-M 方法	11
5.1	设计目标	11
5.2	总体架构	12
5.3	四类逻辑角色	12
5.4	Memory Store 与 MemoryItem	13
5.5	可靠性与完成约束	13
5.6	可审计性	13
6	实验设置	14
6.1	实验矩阵	14
6.2	Hard breadth 完整性	14
7	实验结果	15
7.1	Hard breadth 主结果	15
7.2	r39 与 r56 配对结果	16
7.3	Memory mechanism health	16
8	典型案例与误差分析	16
8.1	成功案例: B2 Calendar	16
8.2	r56 Contacts: 局部 guard 正确, 系统仍然失败	17
8.3	循环检测与恢复脱节	17

8.4	Completion deadlock	17
8.5	H17: 证据权限而不是内容遗忘	17
8.6	自然 memory event 审计	18
8.7	分层错误分类	18
9	讨论：原假设的边界	19
9.1	已经支持的结论	19
9.2	尚未支持的结论	19
9.3	为什么负结果仍然有研究价值	19
9.4	从模块激活到反事实效用：记忆价值链	20
9.5	评测资格阶梯：先证明实验有资格回答问题	20
10	第一阶段两周实验复盘与整体思考	21
10.1	阶段目标：先建立完整且可检验的任务闭环	21
10.2	两周实验如何推进	22
10.3	这一阶段做得比较好的地方	22
10.4	实验暴露出的不足与风险	23
10.5	两周实验如何改变我的研究判断	24
10.6	我现在认为最重要的五个问题	24
10.7	我对“记忆过时”的新理解：三种时钟	25
11	从候选创新到科学终止：最近阶段的研究判断	25
11.1	为什么没有继续给 RAVEN-M 增加模块	25
11.2	如何使用 GPT Pro，而不是让讨论代替证据	26
11.3	被检验的窄假设：Correct Memory, Wrong Target	26
11.4	最接近工作审计：剩余现象究竟属于什么问题	27
11.5	对原方法线的决策：停止扩展，但保留研究价值	28
11.6	最后一次自然错误筛查：反馈已经出现，动作为什么没有改变	28
11.7	为什么这个实验仍然没有被执行	29
11.8	我的最终理解：真实、可检验和创新是三道不同的门	30
11.9	多框架同条件复现：从运行资格走向 Hard pulse	30
11.10	部分 Hard 结果真正告诉了我什么	31
11.11	创新性与完成度边界	32
11.12	对齐官方 Qwen 控制器后的完整 Hard 基线	32
11.13	源材料覆盖诊断：高精度局部记忆仍可能系统性缺项	35
11.14	可执行覆盖门：滚动增加了，但完整信息没有随之出现	36
11.15	最小 L4 救援：一次有价值的负结果	38
11.16	对象角色证据：自然语言规则仍然不够	38
11.17	把 Critic 单独拿出来：终态截图仍不足以证明完成	39

12 有效性威胁	40
13 结论	43
A 代码、配置与复现材料	45
B 考核要求对应关系	48

1 引言

多模态大模型能够同时处理自然语言和屏幕图像，使智能体可以观察手机 GUI、理解用户任务并输出点击、滑动、输入和返回等动作。与单轮视觉问答不同，Mobile-use Agent 必须在连续页面转换中维护任务目标和中间信息：它既要知道当前页面上有什么，也要记住此前完成了什么、读取了哪些值、哪些动作失败、还剩哪些要求未完成。AndroidWorld 提供了 20 个真实 Android 应用中的 116 类可程序化任务，并通过底层状态初始化、成功检查与清理逻辑实现动态评测 [1]，因此适合验证端到端手机控制智能体。

然而，“给模型更多历史”并不自动等于更好的记忆。完整轨迹会快速消耗上下文，自然语言摘要可能丢失字段或把计划写成事实，跨任务经验可能因 App 版本变化而失效，Critic 也可能因为过度保守而阻止正确动作。更关键的是，一条记忆是否可信不能只看它是否曾经出现：同一条“点击右下角加号可进入创建联系人页面”的经验，可以支持低风险导航，但不能单独证明联系人已经保存，也不应无条件迁移到另一 App 或另一版本。

我最初对这个课题的理解比较直接：多模态大模型已经能够“看懂”屏幕，Agent 框架要做的就是为它补上存储、计划和自我检查。但在真正查看轨迹后，我发现这种说法仍然过于粗糙。存储不等于学习，模型输出 done 也不等于任务完成；点击动作被执行、页面发生变化、页面被识别为目标页面、底层任务真实完成，是四个不同层次的命题。这个区分逐渐成为我理解整个项目的起点：记忆系统不仅要保存内容，还要说明内容从哪里来、属于哪个对象、在什么范围内有效，以及它究竟能支持哪一种动作。

第一阶段约两周的框架搭建、系统评测与连续诊断进一步改变了我的判断。开始时我更关注 stale、superseded、contradiction 等长期生命周期问题，因为真实 App 确实可能更新，三个月前的路径也可能失效；但在 AndroidWorld 上完成广度实验、配对实验和连续 revision 诊断后，现有轨迹几乎没有自然产生这类事件。相反，反复出现的是同一 episode 内的字段绑定、目标遗忘、无效动作循环和完成证据被拒。于是我先把问题缩小为“什么证据在什么动作上具有多大权限”，随后又进一步追问：

如果记忆里的值完全正确，错误究竟是记忆造成的，还是模型根本没有把目标文字落到正确的屏幕区域？

这个问题最终没有产生正向方法，而是把原先混在一起的 memory、grounding 和执行问题分开。本文后半部分记录的正是这条“提出窄假设—冻结实验—接受反证—审计创新边界—停止方法线”的研究过程，而不是在失败后继续寻找能让方法显得有效的设置。

本课题要求完成 GUI Agent 记忆机制调研、Qwen3-VL-32B-Instruct AndroidWorld baseline、融合记忆的多智能体框架、Hard task 系统评测、性能比较、案例分析以及代码和方法文档。本文据此围绕以下研究问题展开：

RQ1：不同历史表示能否支持 Qwen3-VL-32B-Instruct 完成 AndroidWorld 长程任务？

RQ2：带 provenance、lifecycle 和 routing 的多角色记忆框架是否优于简单摘要？

- RQ3: 记忆机制是否真实激活, 其额外成本是否转化为成功率?
- RQ4: 失败主要来自记忆, 还是来自视觉定位、动作接口、规划、恢复与完成判断?
- RQ5: 在把成败归因于 memory 之前, 实验必须先通过哪些接口、执行、测量和能力资格门槛?
- RQ6: 当候选机制未通过效应门、剩余现象又被更接近的已有工作覆盖时, 应如何作出停止而非继续包装的研究决策?

本文完成的主要工作如下:

1. 构建冻结的 Qwen3-VL-32B-Instruct AndroidWorld 端到端链路和 B0-B3 历史基线;
2. 设计并实现包含四类角色、四类记忆存储、证据来源和分级路由的 RAVEN-M;
3. 完成 95-cell Hard breadth、r39/r56 配对实验与 H17 连续诊断, 统一记录成功、动作、模型调用、tokens、时间和审计事件;
4. 对负结果进行分层归因, 并提出从实验基础设施到 memory efficacy 的资格阶梯, 避免把结构性零分误解为方法等效或方法失败;
5. 提出并实际检验 *Correct Memory, Wrong Target* 窄假设, 以 96 个条件单元和 192 次模型调用完成 grounding floor/ceiling 诊断, 并依据预注册门槛停止该方法线;
6. 从冻结 Hard 轨迹中筛出高频的 non-progress feedback 失效, 定位 feedback 到 next-action policy update 的 first-broken edge, 并在实现前因最接近工作实质重叠而停止旧方法线;
7. 在不新增方法模块的前提下完成四框架匹配 S1b 资格验证, 并启动预冻结的 Hard pulse; 截至服务器离线完成 16 格中的 8 格、获得 7 个合法分数, 同时把 evaluator 失败、控制器失败和基础设施中断分开记账。

因此, 本文不是一篇声称新方法已经取得正向性能的论文, 而是一份从完整 baseline、复杂框架负结果、因果链诊断到候选假设终止的阶段性研究报告。对夏令营考核而言, 它回答了系统如何搭建和评测; 对科研训练而言, 它更关心两个基础问题: 什么证据有资格支持什么动作, 以及什么时候已有证据已经要求我停止当前方向。

2 相关工作与研究切入点

2.1 端到端 Mobile GUI Agent

AppAgent 通过自主探索或人类演示建立 App 知识库, 使多模态智能体能够像用户一样操作智能手机 [2]。Mobile-Agent-E 进一步使用 Manager、Perceptor、Operator、Action

Reflector 与 Notetaker，并将可跨任务复用的经验组织为 Tips 和 Shortcuts[3]。这些工作说明，手机智能体不仅需要单步视觉理解，也需要规划、执行、反思和经验复用。

AndroidWorld 的价值在于其任务由参数动态生成，并由环境底层状态而非模型自述判断成功 [1]。这使“模型认为自己完成了”与“任务真实完成”可以被严格区分，也是本文坚持采用 native evaluator 的原因。

2.2 GUI Agent 的六类记忆机制

现有 GUI Agent memory 可以概括为六类。

1. **短窗口与完整轨迹**：保留最近若干 action-observation 或截断后的全部历史，容易实现但会带来上下文膨胀。
2. **自然语言摘要**：压缩当前进度、关键值和下一步，成本较低，但可能丢失实体与字段绑定。
3. **结构化状态**：以有限状态机、页面节点或结构化 ledger 表示任务状态。Agent-SAMA 将页面视为状态、动作视为转移，并用四个专门角色支持规划、验证和恢复 [6]。
4. **程序性知识**：保存 App 用法、页面转移和可复用动作序列。PG-Agent 将历史轨迹转换为 Page Graph，并结合检索增强生成支持未见场景 [5]。
5. **跨 session 长期记忆**：MemGUI-Bench 面向跨时间和跨空间保持设计 memory-centric tasks，并以多次尝试评估记忆能力 [7]。
6. **Critic 与验证**：OS-Sentinel 结合 formal verifier 和 contextual judge 检查移动 GUI workflow 风险 [8]；SeerGuard 则在执行前预测动作后果和风险 [9]。

2.3 仅按“长短期”分类的不足

对 GUI Agent 而言，更有解释力的划分是信息在决策链中的功能。表1 展示了七类常见信息及其主要风险。

表 1: GUI Agent 历史信息的功能分类

信息类型	示例	主要风险
Event record	已点击右下角“+”	日志缺失或动作未生效
State transition	已进入新建联系人页面	页面识别错误
Task fact	电话号码为某一值	OCR 或保持错误
Entity binding	该电话属于目标联系人	对象错配
Procedure	“+”可进入创建页	App、版本或页面作用域失配
Goal state	姓名已填，仍需保存	目标漂移或虚构要求
Completion evidence	列表中出现新联系人	假完成或旧数据混淆

这些信息不应共享同一个 confidence 或 lifecycle。本文最初关注通用记忆可靠性，实验后发现更直接的缺口是：历史事实是否与正确实体、字段和视觉来源绑定，以及它在当前动作风险下能够授权什么。

3 任务定义与评测原则

3.1 Agent 交互闭环

在第 t 步，智能体接收任务 g 、当前观察 o_t 和经过策略编译的历史 h_t ，由模型生成动作 a_t ：

$$a_t \sim \pi_\theta(a \mid g, o_t, h_t). \quad (1)$$

动作经 canonical adapter 映射到 AndroidWorld，环境返回下一观察 o_{t+1} 。任务是否完成不由模型自行判断，而由环境 evaluator 读取底层状态：

$$y = \text{Eval}(s_T, g), \quad y \in \{0, 1\}, \quad (2)$$

其中 s_T 是终止时的真实环境状态。该定义区分了动作级验证与任务级完成：某个点击成功引发页面转换，只能证明该动作有效，不能证明整个任务完成。

3.2 动作空间

统一动作包括 open_app、tap、swipe、type_text、press_back、press_home、wait、answer 和 done。坐标使用归一化表示，再由 adapter 映射到设备像素。模型必须输出规范 JSON；无法直接解析时允许一次格式修复，但修复调用同样计入资源消耗。

3.3 公平性与证据等级

所有配对实验固定 AndroidWorld commit、任务实例、seed、模型 revision、BF16 backend、截图处理、生成设置、native action budget、上下文上限与 evaluator，并对所有角色调用进行计费。本文将证据区分为：

- **正式结果**：冻结协议下的有效 scored episodes；
- **开发结果**：用于资格验证和问题定位，不外推普遍效应；
- **观察性审计**：用于提出假设，不能替代盲法标注；
- **未来方案**：尚未执行，不写成已完成结论。

4 AndroidWorld Baseline

4.1 实验环境

表 2: AndroidWorld 与模型环境

项目	配置
Android host	Windows, Python 3.11.9
Android 环境	API 33, AndroidWorldAvd, Pixel 6 profile
注册任务	116
原始截图	2400 × 1080 × 3, 送模前长边缩放至 1280 px
Model host	Ubuntu 20.04.6 LTS
GPU	8 张 RTX 4090, 推理 backend 使用其中 4 张
模型	Qwen/Qwen3-VL-32B-Instruct
Revision	0cfaf48183f594c314753d30a4c4974bc75f3ccb
推理配置	bfloat16, context cap 8192, max new tokens 256
Backend 标识	qwen3_vl_32b_transformers_bf16_4x4090_v1

模型 snapshot 约 66.7 GB。十次 7,704-token 最大形状压力测试全部通过，平均延迟约 10.00 s。真实截图 smoke 中，模型输出打开 Contacts 的动作，prompt 为 2,677 tokens，completion 为 19 tokens，延迟 5.305 s。

4.2 端到端链路

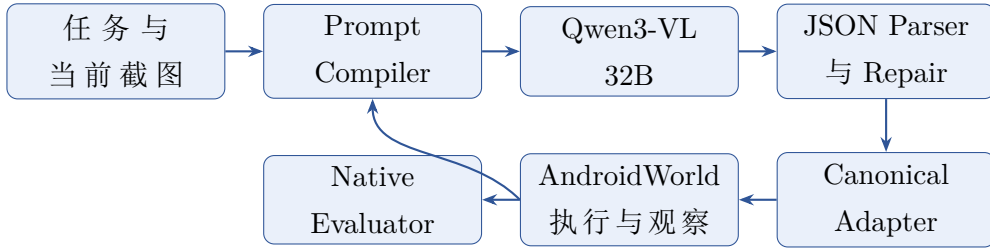


图 1: Qwen3-VL AndroidWorld baseline 的端到端闭环

4.3 历史基线

表 3: 四种历史策略

变体	名称	输入历史
B0	Current Screen Only	仅任务、当前截图、UI tree 和动作规范
B1	Short Window	加入最近 3 个 transitions
B2	Bounded Full History	保留更多轨迹，超过 context cap 时截断
B3	Simple Summary	用自然语言保存进度、页面、已完成步骤和关键值

B3 是本文最重要的强简单基线。只有完整方法在相同任务、seed 和模型条件下超过 B3，才能说明复杂 memory middleware 带来净收益。

5 RAVEN-M 方法

5.1 设计目标

RAVEN-M 是建立在单一冻结模型 endpoint 上的逻辑多角色框架，不训练新参数。其原始目标是：防止未经证实、相互冲突或作用域不匹配的历史直接授权高风险动作和完成声明。

5.2 总体架构

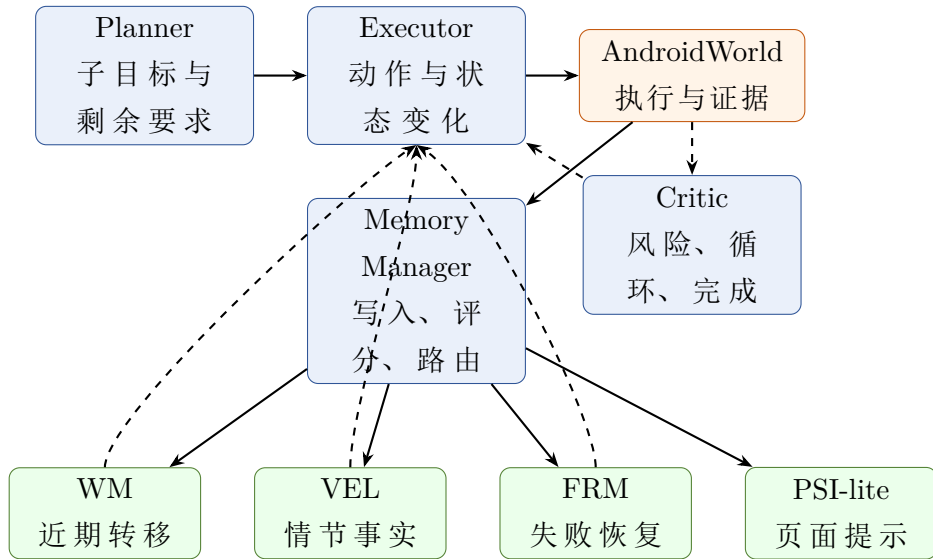


图 2: RAVEN-M 的逻辑角色、环境与记忆存储

5.3 四类逻辑角色

表 4: RAVEN-M 的角色职责与已观察风险

角色	职责	已观察风险
Planner	维护 phase、subgoal、remaining requirements 和恢复方向	可能发明任务未要求的目标
Executor	结合当前屏幕、计划和 memory bundle 输出 canonical action	解释性文本可能被误当事实
Memory Manager	建立 MemoryItem, 更新 lifecycle、scope、score 和 route	确定性流程无法保证语义内容为真
Critic	在循环、冲突、高风险动作和完成时检查	能检测问题但未必产生有效恢复

5.4 Memory Store 与 MemoryItem

表 5: 四类记忆存储

Store	容量	内容	目的
WM	最近 3 个 transitions	短期 action-observation	保持局部连续性
VEL	top 8	episodic facts	保存中间变量和完成证据
FRM	top 2	failures / alerts	避免重复失败
PSI-lite	top 2	episode-local page hints	提供页面与导航假设

每条 MemoryItem 记录 episode、task、类型、subject-predicate-object、状态、源 observation/action/model call、截图路径与 SHA-256、页面签名、作用域、前置条件、关系和路由历史。生命周期包括 candidate、observed、verified、stale、contradicted、revoked、superseded 与 archived; 运行时 route 分为 FACT、HYPOTHESIS、ALERT 和 SUPPRESS。

5.5 可靠性与完成约束

原方法依据 verification status、source quality、recency、page compatibility、contradiction 与 lifecycle 计算可靠性。可抽象为:

$$R(m, t) = \sum_i w_i \phi_i(m, t), \quad (3)$$

其中 ϕ_i 是来源、时效、页面兼容和冲突等特征。但实验表明, 单一标量难以同时处理低风险导航和高风险完成: 84% 被引用记忆为 HYPOTHESIS, 而 terminal completion 又要求严格 FACT, 造成中间动作与终止权限不一致。

M0 的 done 需要 supporting evidence、任务要求覆盖和必要的 Critic 授权。该设计可以降低假完成, 却产生了显著 false rejection: 五个 M0 episodes 中 13 次 completion attempt 全部被拒绝。

5.6 可审计性

G6 验证中, 75 个 full tests 和 45 个 targeted tests 通过, 20 个 corruption fixtures 均未被 route 为 FACT; cross-episode 读写被拒绝, 截图 hash、证据绑定与 append-only replay 可检查。这证明 memory middleware 的工程机制确实存在, 而不是 prompt 中的概念描述。

6 实验设置

6.1 实验矩阵

实验分为三层：

1. **Hard breadth**: 19 个 Hard tasks, B0/B1/B2/B3/M0 五种变体, 共 95 个 cells;
2. **Gate E**: 在相同四类 non-Hard 任务和 seed 上比较 B3 与 M0, 用于开发资格验证;
3. **H17 连续诊断**: 分析同一复杂信息检索任务中的 evidence binding、guard、repair 与 terminal authority, 不作为未见泛化结果。

主指标为任务成功率：

$$\text{SuccessRate} = \frac{1}{N} \sum_{i=1}^N y_i. \quad (4)$$

同时报告 actions、model calls、prompt/completion tokens、episode wall time、model latency、loop、no-screen-change、memory route 和 completion rejection。相对开销定义为：

$$\Delta C(M0, B3) = \frac{C_{M0} - C_{B3}}{C_{B3}} \times 100\%. \quad (5)$$

6.2 Hard breadth 完整性

表 6: Hard breadth 完整性与资源统计

检查项	结果
Scheduled / completed	95 / 95
Valid scored episodes	95
Five-variant task pairs	19 / 19
Pairing errors	0
Episode audit errors	0
Overall success	1 / 95
Model calls	4,021
Total tokens	17,971,842
Episode wall time	22.01 h
Model latency	13.22 h

其中 15 个 information-retrieval cells 因冻结动作 schema 缺少 `answer` 而结构性无法得分。本文没有事后删除这些样本；同时给出排除后 supported-task success 为 1/80，以区分接口缺陷和方法能力。

7 实验结果

7.1 Hard breadth 主结果

表 7: 19 个 Hard tasks 上的五变体结果

Variant	Success	Mean calls	Mean tokens	Mean wall time
B0	0/19	35.53	127,873	10.84 min
B1	0/19	38.84	168,683	12.45 min
B2	1/19	35.95	193,415	12.04 min
B3	0/19	43.47	180,404	13.57 min
M0	0/19	57.84	275,511	20.61 min

唯一成功来自 B2 的 `SimpleCalendarAddOneEvent`。M0 相对 B3 的 model calls 增加约 33%，tokens 增加约 53%，wall time 增加约 52%，但成功率没有提高。因此 Hard breadth 支持“完整方法真实运行且成本更高”，不支持“复杂 memory 优于简单摘要”。

表 8: Hard breadth 终止结果

Outcome	Count
Budget exhausted	53
Model infeasible	21
Premature completion	19
Invalid output	1
Success	1

所有方法接近 capability floor，意味着该结果不能精确估计小幅 memory effect。零附近的成功率既不是“memory 一定无效”的证据，也不能被解释成已获得正向改进。

7.2 r39 与 r56 配对结果

表 9: r39 与 r56 的 B3-M0 配对汇总

Run	Arm	Success	Mean actions/steps	Mean calls	Prompt tokens	Completion tokens
r39	B3	4/4	10.25	14.00	67,920.25	1,225.00
r39	M0	3/4	11.75	20.00	120,122.50	2,415.50
r56	B3	4/4	9.25	14.25	72,817.50	1,264.25
r56	M0	3/4	11.75	19.75	123,449.00	2,331.25

r56 中, M0 相对 B3 的 actions 增加 27.03%, calls 增加 38.60%, prompt tokens 增加 69.53%, completion tokens 增加 84.40%; 成功率由 100% 降至 75%, 即绝对下降 25 个百分点。四任务规模不足以证明普遍效应, 但足以说明当前实现没有出现稳定的正向信号。

7.3 Memory mechanism health

表 10: Hard M0 的记忆机制激活情况

指标	数量	指标	数量
Decisions receiving bundle	758	Decisions citing memory	457
History/Planner/Critic calls	270	FACT routes	150
HYPOTHESIS routes	2,777	ALERT routes	105
SUPPRESS routes	5,684	Loop events	46
Memory audit failures	0/19	Completion attempts rejected	13/13

这些数据排除了“memory path 是 dead code”的解释, 但也说明 mechanism activity 不能替代 task utility。路由、引用和审计都发生了, 额外信息仍未转化为成功。

8 典型案例与误差分析

8.1 成功案例: B2 Calendar

Hard breadth 的唯一成功是 B2 在 SimpleCalendarAddOneEvent 上以 21 个动作获得 evaluator success。这说明模型与环境并非完全不可用, 受限历史能够支持至少一个复杂任务; 但单次成功不具有统计意义, 也不能推出 B2 普遍优于其他变体。M0 在同一任务上重复点击 time picker 并耗尽预算, 显示恢复而非存储容量是关键差异。

8.2 r56 Contacts: 局部 guard 正确，系统仍然失败

M0 正确填写了姓名和电话，但 Planner 发明任务未要求的 Company 字段。guard 正确阻止了虚构 company，随后系统却没有回到“直接保存”的可执行路径，最终 evaluator failure。该案例表明，局部安全判断正确不等于系统整体正确：requirements 必须由任务解析器封闭产生，guard 拒绝后还必须返回可执行替代动作。

8.3 循环检测与恢复脱节

在 80 个受支持 cells 中，59 个至少出现三次相同动作，32 个至少出现十次，累计 477 次动作后截图不变。46 次 loop critic 全部给出 **reobserve**，其中有后继动作的 44 次里 14 次立即重复原动作。问题不在于系统没有发现循环，而在于 detection 没有改变下一步动作权限。更直接的机制应记录 state-action 对，在连续无效果后临时屏蔽相同动作类，并要求选择不同恢复方向。

8.4 Completion deadlock

五个 M0 episodes 中 13 次 completion attempt 全部被拒。原因包括 evidence 写入后没有及时 route 为 FACT、Executor 仍引用旧 HYPOTHESIS、Critic 和 validator 的权限标准不一致。统一全局 route 无法表达“哪个任务要求已经由哪条证据覆盖”，因此应改为 requirement-wise closure。

8.5 H17: 证据权限而不是内容遗忘

表 11: r76-r78 H17 连续诊断

Run	Success	Actions	Calls	Wall time	核心结果
r76	0	16	31	1,233.745 s	两个字段已找到，但 evidence contract 拒绝回答
r77	1	13	23	560.688 s	reward 1.0，但最大 prompt 为 8,982
r78	0	7	13	610.339 s	repair 坐标落在两行之间，未进入目标行

r76 已获得两个目标 activity type，却因 visit key、MemoryItem ID 和 screenshot authority 混淆而无法回答。r77 证明 same-episode entity-bound visual evidence 可以支持 terminal answer，但该任务已被反复开发，最大 prompt 超过 8,192，而且 terminal 没有引用 MemoryItem。r78 相邻 revision 再次失败，说明 r77 只是窄成功，不能证明 lifecycle 或未见泛化。

8.6 自然 memory event 审计

23 个 Hard M0 memory logs 共包含 10,884 个事件、876 次写入和 15 个 contradiction flags。人工观察显示，其中 14 个更像兼容转述，只有 1 个较实质污染：系统预期打开新消息编辑页，但实际仍在短信主页，expected outcome 被写成 observed fact。与此同时，stale、superseded、revoked、archived 和 invalidated 等自然事件没有形成有效样本。这说明当前 benchmark 更适合研究同 episode 信息保持，而不适合直接证明跨月、跨版本的长期过时问题。

8.7 分层错误分类

表 12: 失败类型、例子与 memory-specific 判断

Family	示例	是否 memory-specific
Interface	冻结 schema 缺少 answer	否
Perception	控件漏检、符号误识别	否
Grounding	坐标落在相邻控件之间	否
Planning	Planner 发明 Company requirement	与 goal state 相关
Loop	相同无效动作重复	与 recovery 相关
Creation	expected outcome 写成 observed fact	是
Binding	值绑定到错误 row 或实体	是
Retention	后续步骤丢失已读字段	是
Completion	正确 evidence 被 validator 拒绝	与 authority 相关
Context	prompt 超过 cap	否
Staleness	App 更新导致旧路径失效	当前未自然观察

根据现有频率和影响，修复优先级为 action/interface、perception/grounding、loop/recovery、planning/goal、completion authority、context、memory creation/binding，最后才是 natural staleness。

9 讨论：原假设的边界

9.1 已经支持的结论

现有证据支持以下主张：baseline 和完整方法均真实运行；模型调用、tokens、动作与时间 accounting 完整；memory routing 确实激活；provenance、cross-episode isolation 和 corruption invariants 工作；guard 能阻止部分模糊动作；same-episode entity-bound visual evidence 具有潜力。

9.2 尚未支持的结论

现有证据不支持：M0 优于 B3、完整 lifecycle 提高成功率、Critic 能有效恢复循环、completion gate 净收益为正、H17 能泛化到未见任务、系统已经实现长期学习。表13 将结论与证据边界明确分开。

表 13: 主张、当前证据与正确表述

主张	当前状态	正确表述
方法已完整实现	支持	代码、审计和真实运行均可检查
Memory route 真实激活	支持	758 次接收 bundle, 457 次引用
M0 提升 Hard 成功率	不支持	B3 与 M0 均为 0/19
M0 提升效率	反证	calls、tokens 与 wall time 均增加
Lifecycle 是主瓶颈	不支持	自然 stale/superseded 等事件不足
Entity-bound evidence 有潜力	初步支持	H17 窄案例，仍需 held-out 检验
资格阶梯可定位失败层	支持	五轮均能给出首个未通过接口
Action-conditioned oracle 提高准确率	未测试	v0.2.3 正式 held-out trace 为 0

9.3 为什么负结果仍然有研究价值

一次实验不能证明普遍规律，但冻结协议、完整 accounting 和逐层归因可以排除许多含糊解释。本文没有在看到结果后删除困难任务，也没有只报告 r77 的成功。相反，连续记录揭示了一个更重要的问题：如果 benchmark 不自然产生 memory lifecycle hazard，那么继续增加 stale、supersede 和 contradiction 规则，只会让 controller 越来越像针对少数任务的硬编码系统。

这个结果也改变了我对 guard 的看法。最开始我认为 guard 写得越完整，系统就越安全；但实验说明，guard 只负责阻止错误还不够。如果它拒绝动作以后不能给出新的可

执行路径，就会把“局部正确”变成“全局失败”。另一方面，必要规则确实存在，例如滑动方向必须与语义一致、不可逆动作需要确认、任务完成必须检查底层状态。真正困难的不是要不要规则，而是如何区分可迁移的系统不变量与针对某个任务反复调出来的特殊分支。我认为这个边界只能通过 held-out 任务、negative controls 和删除模块后的消融实验来证明，不能靠继续增加规则数量来证明。

因此，当前最合理的转向不是继续扩大框架，而是先证明某类 memory-specific error 在目标任务中真实存在，再用最小机制解决它。

9.4 从模块激活到反事实效用：记忆价值链

我进一步认识到，“memory 有用”至少包含六个依次相接的条件：**Creation**，信息来自真实观察而不是模型预期；**Binding**，值与正确实体、字段和来源绑定；**Retrieval**，在真正需要时被取回；**Authorization**，证据只授权其作用域和风险等级允许的动作；**Execution**，动作通过统一接口并在环境中真实生效；**Verification**，动作结果和任务闭环被适当的观测或 evaluator 确认。对一次具体的 evidence-action 使用，可以写成诊断量

$$Q(m, a) = C(m) B(m) R(m, a) A(m, a) X(a) V(a), \quad (6)$$

其中每一项表示相应环节是否成立。这个乘积不是新的训练目标，也不能直接等同于任务成功率；它的作用是提醒我，只要任一上游环节为 0，下游“memory 方法是否更好”的比较就会被更早的失败截断。现有 M0 health 指标主要证明了 Creation、部分 Retrieval 和审计机制发生过，却没有证明完整的 $Q(m, a) = 1$ 链条，更没有证明相对 B3 的净收益。

真正的 memory utility 必须是反事实的：在相同任务、seed、模型和资源约束下，使用该机制相对于匹配成本的基线是否改变了决策并提高了 evaluator success。因而本文不把 route count、引用次数或单条记录准确率称为 efficacy，而把配对成功差异与额外 calls、tokens、时间和 false rejection 一起报告。机制指标用于定位因果链断在哪里；只有 held-out paired outcome 才能回答这条链是否产生净价值。

9.5 评测资格阶梯：先证明实验有资格回答问题

连续失败使我形成了一个比继续调 controller 更重要的实验原则：**memory efficacy** 是资格阶梯的最高层，而不是所有运行都能直接计算的指标。表14 给出本文当前采用的六层顺序。

表 14: 从实验基础设施到 memory efficacy 的资格阶梯。更低一层未通过时，不解释更高层效应。

层级	资格问题	最低证据	本文暴露的例子
L0	基础设施与账本	reset、调用计数、完整 trace、失败也原子落盘	v0.2.3 pre 证据与 completion record 失败
L1	Decision contract	模型输出、schema、normalizer、adapter 单一一致	v0.2 非 canonical action; v0.2.1 metadata 拒绝
L2	动作真实执行	合法动作到达环境且仅执行一次	v0.2.2 为 3/3
L3	结果可测量	与动作类型匹配的 stable outcome evidence	v0.2.2 PressBack 被 pixel 条件拒绝
L4	基础任务能力	强简单基线脱离 capability floor	Hard breadth 接近 0 成功
L5	Memory efficacy	新 held-out 配对、强基线、成本与误差共同报告	当前尚未获得资格

这个阶梯改变了我对“失败实验”的解释方式。v0.2 的 0/9 不是三种 memory arms 等效，因为它只到达 L1；v0.2.2 的 2/3 也不是 memory 提升，因为它只在检查 L2-L3；v0.2.3 更不能否定 outcome oracle，因为正式 held-out trace 尚未形成。只有把失败定位到最低未通过层，并停止解释其上的指标，才能避免用大量无效运行制造看似丰富、实则无法回答研究问题的数字。

10 第一阶段两周实验复盘与整体思考

10.1 阶段目标：先建立完整且可检验的任务闭环

第一阶段并不是直接追求一个复杂的新方法，而是先把 Mobile-use Agent 从接收任务到获得真实 reward 的整条链路跑通。我最初用创建联系人这样的任务理解 Agent 到底缺什么：一个视觉大模型可以识别“+”、输入框和 Save，但它不会自动形成完整任务闭环；框架还需要保存任务、拆解计划、记录页面转换、维护关键字段，并在最后调用 evaluator 检查真实状态。这个起点让我明确了三个最基础但也最容易混淆的概念。

1. **Memory 是保存，不等于学习。**一次轨迹被写入 store，只能说明系统能够在以后取回它；只有当这些信息改变后续决策并在新任务上产生可验证收益时，才能讨论学习或经验复用。
2. **动作正确不等于任务完成。**点击“+”后进入 Create Contact 页面，可以验证这一步 transition；保存联系人则必须在列表或底层数据库中看到目标实体和字段，不能只看 Save 被点击。

3. **当前观察与历史证据具有不同权威。**当当前截图和旧记忆冲突时，低风险导航应优先相信当前页面；但网络故障、加载延迟和 App 异常也可能使一次观察不可靠，因此不能因为一次失败就永久删除旧经验，而应标记不确定、重试并保存作用域。

我进一步把“点击 + 后进入创建页面”拆成四个命题：Agent 执行了 tap；前后 observation 发生变化；新页面在语义上是 Create Contact；联系人已经成功保存。前两项可以由日志直接确认，第三项需要视觉语义判断，第四项必须由任务级闭环确认。这个分层让我意识到，不应该因为最终完成需要严格验证，就反复怀疑已经由连续两帧确认的低风险页面转换。

10.2 两周实验如何推进

这两周的工作可以分为四个连续层次。首先，我完成 Qwen3-VL-32B-Instruct、AndroidWorld、模拟器和 evaluator 的端到端部署，并建立 B0-B3 历史基线，使模型动作、环境状态和最终 reward 能够统一记录。其次，我加入 Planner、Executor、Memory Manager 与 Reflector/Critic，形成 M0 框架，并通过 provenance、memory status、routing 和审计事件检查机制是否真正被调用。再次，我在 19 个 Hard tasks、5 种设置上完成 95-cell breadth，对成功率、模型调用、tokens、时间、循环和无屏幕变化动作进行统一统计。最后，我利用 r39、r56 配对结果和 r61-r78 的 H17 连续诊断追查具体失败链，而不是只停留在最终 reward。

实验推进的同时，我把直觉与已有工作对照。AppAgent 和 Mobile-Agent-E 说明程序性经验和跨任务记忆可以减少重复探索 [2, 3]；Agent-SAMA 和 PG-Agent 说明 FSM 与 Page Graph 可以表达页面结构 [6, 5]；MemGUI-Bench 说明真正的长期记忆需要跨时间、跨空间和多次尝试的专门评测 [7]；OS-Sentinel 与 SeerGuard 则提醒我，不可逆动作的风险验证与普通导航不是同一问题 [8, 9]。这些文献为框架设计提供了参照，但是否适用于当前 benchmark，仍然必须由本项目的真实轨迹回答。

10.3 这一阶段做得比较好的地方

第一，完整链路已经能够稳定运行，最终结果由 AndroidWorld 原生 evaluator 检查，而不是由模型自行宣布完成。第二，实验没有只保留成功案例：95 个 Hard cells、r39/r56 和 H17 连续 revision 都保留了动作、调用、tokens、时间与审计记录，使失败可以被定位到感知、定位、动作格式、状态变化、循环恢复或完成判断。第三，M0 的 memory route、角色调用、FACT/HYPOTHESIS/ALERT/SUPPRESS 与 completion rejection 均可计数，因此可以区分“模块确实运行”与“模块带来任务收益”。第四，连续诊断没有隐藏 r76 和 r78 的失败而只报告 r77 的成功，这使单例修复、重复调试和域内过拟合之间的边界能够被公开讨论。

这些工作说明第一阶段达成了工程和研究上的基本目标：不仅把 baseline 和记忆框架跑起来，还建立了一条可以审计、比较和复现的证据链。对于夏令营任务而言，这是完

整交付的基础；对于后续研究而言，它更重要的作用是暴露原始假设与真实失败分布是否匹配。

10.4 实验暴露出的不足与风险

最明显的问题是任务成功率仍然很低。95-cell Hard breadth 只有 1 次成功；在 B3 与 M0 的直接比较中，复杂记忆框架没有带来成功率提升，却显著增加调用、tokens 和运行时间。其次，框架最强调的 stale、superseded、revoked 等 lifecycle 事件在自然轨迹中几乎没有出现，说明当前 benchmark 未必能检验这部分设计。再次，Critic 和 guard 有时能够发现风险，却经常不能在拒绝后提出有效替代动作；“检测正确”没有闭合为“恢复成功”。最后，H17 经历大量连续 revision 后仍表现不稳定，说明反复观察相同任务会逐渐把 controller 推向任务特定规则，域内成功不能直接作为泛化证据。

最让我改变想法的并不是某一个成功数字，而是三组相互矛盾的现象：memory route 大量激活但成功率没有增加；guard 多次正确阻止模糊动作但系统未能恢复；r77 获得 reward 1.0，紧邻的 r78 又因为坐标落在两行之间失败。这些现象说明，机制“运行了”与机制“有净价值”必须分开，单个成功案例也只能证明一条可能的机制链，不能证明普遍泛化。

在完成内部审计后，我又引入一次独立外部审阅，要求对项目作出 GO、REVISE、PIVOT 或 STOP 判断。外部审阅最终建议 PIVOT：保留 EventLog、entity-field evidence、GoalLedger、Recovery Registry 和可审计性，把 periodic Planner、通用 Critic、PSI-lite、全局 reliability formula 与完整 lifecycle 移出 AndroidWorld 主执行路径。这个意见并不是替我决定方向，而是迫使我把自己的判断写成 claim-evidence 对照：哪些已经被数据支持，哪些只是合理直觉，哪些必须换 benchmark 才能研究。

10.5 两周实验如何改变我的研究判断

表 15: 第一阶段两周实验前后研究判断的变化

最初直觉	看到的证据	当前判断
记忆越完整, 任务越稳定	M0 成本显著增加, 成功未提高	先证明信息确实需要被保留, 再选择最小表示
更多 Critic/guard 会更安全	46 次 loop critic 未恢复; 13 次完成全部拒绝	检测必须与恢复和动作权限绑定
Lifecycle 是主要创新点	自然 stale、superseded、revoked 几乎未出现	在 AndroidWorld 中转向 episode-local evidence binding
一次 reward 1.0 可以证明修复有效	r77 成功但 r78 再次失败, 且 H17 已被反复调试	单例只作 case study, 效能必须用 held-out paired test
Route active 说明 memory 有用	758 次 bundle、457 次引用仍未产生净收益	机制健康和任务效用必须分开报告
规则越多越能覆盖异常	controller 逐渐出现任务模式分支	只保留通用不变量, 其余必须通过消融与域外测试

10.6 我现在认为最重要的五个问题

第一, 记忆错误的分母是什么? 只统计 contradiction 次数没有意义, 还需要统计本来有多少个可能发生保持、绑定、覆盖和完成错误的 decision opportunities。没有这个分母, 就无法判断 memory reliability 是否真是主要瓶颈。

第二, 阻止错误之后是否恢复成功? guard 被触发只能说明检测器工作; 只有“阻止错误动作-选择替代路径-最终 evaluator success”闭环成立, 才说明它对系统有净贡献。

第三, 当前截图和历史应该怎样组合? 我的答案不是永远相信其中一个, 而是让当前 observation 对当前页面拥有更高权威, 让历史 evidence 保存跨页面字段; 当 scope、entity 或 state epoch 不匹配时再降级和验证。

第四, 什么才算通用原则? 我认为规则可以来自实验发现, 但必须能用任务无关语言描述, 并在 held-out App、layout 和 seed 上继续成立。只解释 H17 日期列表或某个按钮坐标的规则应当进入 regression fixture, 而不是论文方法。

第五, 创新是否一定来自增加模块? 第一阶段两周实验带给我的最大认识是, 问题重定义也可以是研究贡献。把“普遍可靠的长期记忆”缩小为“实体绑定证据如何授权具体动作”, 虽然架构更小, 却更容易形成明确假设、强对照和可复现实验。

10.7 我对“记忆过时”的新理解：三种时钟

最初我把 stale 主要理解为“三个月前的 App 路径因为版本更新而失效”。实验后我发现，GUI Agent 中至少有三种不同速度的时钟；把它们全部塞进一个 recency score，会把性质不同的问题混在一起。

表 16: GUI Agent 证据的三种时间尺度及其处理原则

时间尺度	典型变化	旧证据仍可保留什么	首选处理
UI state clock	秒级加载、弹窗、滚动、页面切换	已执行动作和已读取字段	当前观察优先；稳定采样与有限重试
Task/entity clock	子目标推进、目标行或账户改变	来源明确的历史值与动作日志	使用 entity、field、phase 和 state epoch 限定作用域
App/version clock	跨 session 更新、布局或首次启动流程变化	旧 procedure 作为待验证候选	保留而不盲删；重新探测后 supersede 或缩小 scope

这一区分解释了为什么“相信当前截图”和“不要删除旧记忆”并不矛盾。当前观察对当前 UI state 拥有最高权威；一条三个月前的 procedure 即使不再能直接授权动作，仍可以作为探索起点。只有在相同适用条件下获得稳定反证，或者新 procedure 在同一目标上被验证成功，旧记录才应被 supersede；一次网络错误或加载失败只能产生 uncertainty，不能永久判死。AndroidWorld 当前主要暴露前两种时钟，第三种时钟需要 cross-session 或 controlled drift benchmark 才能有效研究。

11 从候选创新到科学终止：最近阶段的研究判断

11.1 为什么没有继续给 RAVEN-M 增加模块

第一阶段的负结果留下了一个很容易走偏的选择：继续补充 memory 状态、Critic 和 guard，直到某几个任务成功；或者先判断 AndroidWorld Hard 中到底有没有一个频繁、可隔离、确实由记忆导致的错误。前一种做法工程上可能继续前进，却很难说明新增规则是在解决通用问题，还是逐渐记住了四类反复观察的开发任务。我因此选择后一条路：不再从“我能加什么模块”出发，而从“现有轨迹中哪一种错误能够被最小干预单独改变”出发。

这也修正了我此前对“记忆只做它有资格做的事情”的理解。这个原则本身并不新颖；真正可能形成研究问题的部分，必须落到一个自然发生、能够计数的错误机制上。例如，一条正确的 source fact 若在模型尚未确定 destination 之前进入上下文，是否会使模型把事实所属对象误当成动作目标？若该错误存在，才有理由设计先定位 destination、再

释放 source value 的门控；若错误根本不存在，直接实现门控只是在增加一个看似合理但没有因果对象的模块。

11.2 如何使用 GPT Pro，而不是让讨论代替证据

最近阶段我多次使用 GPT Pro 对完整项目材料进行独立规划和反向审查。它的作用不是替我生成一个更复杂的故事，而是把我已经做过的代码、冻结结果、失败轨迹、污染边界和最接近文献放到同一张证据表上，并明确给出继续、转向或停止的门槛。为了避免只给审阅者看有利片段，后一次审阅直接提供完整仓库材料和三轮原始结果；执行端只负责冻结配置、运行实验和核对产物，不再擅自二次设计一个“更容易成功”的版本。

我认为这种协作方式真正有价值的地方是减少自我确认：先把“什么结果算支持、什么结果必须停止”写清楚，再允许实验说服自己改变判断。它也暴露了本阶段的一个效率问题：此前有些轮次过度停留在基础设施资格和重复修补上，虽然保证了证据边界，却延迟了最小机制实验。更合理的原则不是放弃严谨，而是让严谨服务于最短的判别路径：只修复会阻断当前假设检验的基础设施；一旦最小实验已经越过资格门，就立即看效应；效应门失败后，不再通过改模板继续挽救。

11.3 被检验的窄假设：Correct Memory, Wrong Target

候选假设可以用一句话概括：

记忆里的值可能完全正确，但如果它在目标控件尚未被可靠定位时进入推理，模型仍可能把正确值作用到错误对象上。

实验采用 Timing×Ambiguity 的 2×2 设计。Early 条件在 destination grounding 前提供完全正确的 source fact，Late 条件在 grounding commitment 后提供同一事实；Low/High Ambiguity 控制候选目标的视觉歧义。每个匹配对固定同一张真实 AndroidWorld 截图、同一任务、同一模型 revision、两次模型调用与动作预算，并按真实 Qwen tokenizer 使 Early/Late 文本 token 总数完全相等。冻结模型为 Qwen/Qwen3-VL-32B-Instruct，revision 为 0cfaf48183f594c314753d30a4c4974bc75f3ccb。主要指标不是模型能否复述 source value，而是第一次目标动作是否指向错误对象。

三轮均为开发期机制筛查，不是 held-out confirmatory study；它们合计包含 96 个条件单元和 192 次模型调用，无选择性重试，也没有执行 live Android 动作。结果见表17。

表 17: Correct Memory, Wrong Target 三轮开发期因果筛查

轮次	候选目标信息	低歧义准确率	高歧义错误率 Early / Late	判定
v0.1	target ID、屏幕文字、人工视觉提示、bounds	100%	0% / 0%	grounding ceiling; 交互为 0
v0.2	target ID、bounds; 模型自行完成文字到区域映射	25%	87.5% / 87.5%	grounding floor; 不具备 timing 解释资格
v0.3	target ID、屏幕原有文字、bounds; 无人工视觉提示	100%	0% / 0%	资格通过, 但效应门失败

三轮每轮 32/32 单元有效, parser failure 均为 0, exact-value recall 均为 100%, Timing×Ambiguity 交互均为 0。v0.2 的高错误率表面上最像“正确记忆、错误目标”, 但它在低歧义条件下也只有 25% 准确率; 逐例查看又发现, 第一次 grounding 一旦选错匿名区域, 第二次调用只是忠实延续这个错误 commitment, Early 与 Late 并没有分离。因此它测到的是文字与候选区域之间的映射困难, 而不是正确事实出现得太早。v0.3 恢复屏幕本来就有的文字标签、但不恢复人工语义提示后, 基础定位恢复为 100%, 高歧义条件也全部选对。由此得到的边界非常清楚:

目标能被可靠定位时, 正确事实出现的早晚没有造成可观察的错目标; 目标不能被可靠定位时, 错误由 grounding 主导, 仍与事实出现时机无关。

按照实验前冻结的停止规则, Stage 1 的效应门没有通过, 因此不扩大到正式样本, 不实现 Destination-First Binding Gate, 也不在看过结果后修改同一批模板并把重跑称为 held-out。这个结论只适用于当前 8 个开发模板、3 个 App 场景和冻结模型 revision, 不能外推为“所有 GUI Agent 都不存在 timing effect”; 但它足以否决当前证据下继续开发该方法的理由。

11.4 最接近工作审计: 剩余现象究竟属于什么问题

假设失败后还不能马上把 v0.2 重新命名为一个新贡献, 因为“模型知道文字却点错屏幕区域”首先是标准 GUI grounding 问题。SeeClick 已将 GUI grounding 明确定义为根据指令定位屏幕元素, 并展示 grounding 能力与下游 GUI 任务表现之间的联系 [10]。更接近的是 2026 年预印本 *Do GUI Agents Believe Their Eyes?*: 它直接构造像素与结构信息冲突及自然过时状态, 诊断 GUI Agent 对不同观察通道的依赖, 并在真实交互环境中追踪错误动作和任务失败 [11]。这使“跨模态信息不一致时应相信什么”本身也不再是空白。

同样, 不能只看最终 reward、需要检查中间过程, 也已有 ProBench 的 process-aware evaluation[12]; 结构化轨迹、Page Graph、Critic 和 verifier 在前述工作中也各有覆盖。因

此，若没有一个与这些工作实质不同、且在自然 AndroidWorld Hard 轨迹中具有足够发生率的干预变量，继续把 grounding、history、critic 和 verifier 重新组合并不足以形成强创新。

这次审计改变了我对创新的判断标准。我不再把“过去没有用这个名字组合过”当作创新，而要求同时满足四点：真实错误在目标场景中自然发生；方法改变的是该错误的首个断点；强基线和资源成本匹配；与最接近工作存在可被实验验证的差异。Timing 候选未通过效应门，剩余 grounding 解释又在最接近工作中缺少清楚余量。

11.5 对原方法线的决策：停止扩展，但保留研究价值

截至本报告更新，决策为 **STOP_RESEARCH_METHOD**，后续仅做 scientific closeout：冻结原始结果、核对哈希与污染标签、整理复现材料和负结果边界，不再运行新模型调用、AndroidWorld 批次或方法扩展。这个决定不是说整个夏令营“白做了”，也不是证明 memory 对 GUI Agent 没有价值；它只表示当前 RAVEN-M 及其后续 timing 分支没有获得足以继续包装成新方法的证据。

我认为这段试错真正留下了三点可复用认识。第一，复杂系统的负结果必须找到 first-broken edge，否则 task failure 无法归因于 memory。第二，正确记录可以既没有帮助也没有伤害；只有它在匹配成本的反事实实验中改变动作和最终成功，才能称为 memory utility。第三，科研中的停止规则不是保守，而是对时间成本负责：当资格已通过、效应仍为零、最接近工作又覆盖剩余现象时，继续加模块比接受结论更容易，但科学价值更低。

因此，本项目最终最有价值的“自己的思想”不是一个未经验证的新英文模块名，而是一套逐渐形成的判断方法：从真实失败分布出发，把记忆、grounding、执行和验证分层；用最小干预寻找因果对象；让外部审阅挑战自己的解释；最后允许数据否决最初看好的方向。RAVEN-M 仍可作为工程 baseline、负结果档案和可复现案例保留，但不再被表述为已获得正向方法创新。

11.6 最后一次自然错误筛查：反馈已经出现，动作为什么没有改变

停止 timing 分支以后，我没有立即要求再生成一个“新方法”，而是回到最早的 95-cell Hard breadth，寻找同时具有自然发生、明确分母和较高频率的失败。唯一通过这一步筛查的窄问题是：controller 已经检测到 action loop 或 no-progress，并向模型发出 reobserve，但该反馈没有约束下一步策略，模型随即再次执行完全相同的动作。

冻结 forensic report 给出了两种粒度的证据。80 个动作接口受支持的 cells 中，59 个至少连续重复同一动作 3 次，32 个至少重复 10 次；M0 共触发 46 次 loop critic，其中 44 次能够观察后续动作，14 次立即重复原动作。表18 同时给出普通 Wilson 区间，用于描述该冻结 suite，而不是声称跨任务总体比例。

表 18: 冻结 Hard suite 中循环与反馈不生效的自然发生率

现象	分子/分母	比例	95% Wilson 区间
连续同一动作至少 3 次	59/80 cells	73.75%	[63.18%, 82.14%]
连续同一动作至少 10 次	32/80 cells	40.00%	[29.96%, 50.95%]
Loop feedback 后立即重复原动作	14/44 events	31.82%	[20.00%, 46.56%]

这个现象比“Agent 容易循环”更具体。Detector 和 feedback channel 并非缺失，真正的 first-broken edge 是

$$\text{recovery feedback} \not\Rightarrow \text{next-action policy update.} \quad (7)$$

从纯因果设计看，可以固定 task state、loop event、detector output、base model、history 和资源预算，只改变下一动作是否允许重复：soft feedback 只提出建议；等成本中性条件增加相同思考预算但不给恢复指令；hard binding 则暂时排除已经确认无效的 exact action。主要行为指标是 immediate exact repeat，最终指标仍必须是 AndroidWorld native task success。这样的三臂对照能够区分“模型忽略反馈”“收益只来自额外思考”“重复动作其实是必要等待”以及“禁止重复只是把循环转移成其他无效动作”等解释。

11.7 为什么这个实验仍然没有被执行

一个实验能够识别机制，并不等于它值得被包装为新的研究贡献。VLAA-GUI 已经针对重复失败和持续页面状态设计 mandatory Loop Breaker：切换交互方式、强制改变策略，并把反思信号绑定到后续策略变化；其闭环实验同时报告任务成功、循环和浪费步骤 [13]。在移动端，VeriGUI 已经显式验证动作结果、检测失败并引导修正推理 [14]；BacktrackAgent 也使用 verifier、judger、reflector 和 backtracking 完成错误检测与恢复 [15]。因此，本项目剩余的差异主要是 AndroidWorld、Qwen3-VL-32B-Instruct、本地 reobserve scaffold，以及一个可能更干净的 soft-versus-hard ablation。这些差异具有复现和工程迁移价值，却没有形成新的 causal variable、observation regime 或 intervention class。

基于这一操作层面的重叠，最后一轮独立审阅给出 **NO_GO_NEW_EXPERIMENT**。这次停止发生在代码实现和服务器调用之前：新任务数、模型调用数、AndroidWorld 动作数和基础设施修复次数均为 0。对我而言，这比跑出一组“hard binding 能减少重复”的正向数字更诚实，因为那组数字即使成立，也只能证明已有思想在当前 controller 上同样有效，不能回答新的科研问题。

11.8 我的最终理解：真实、可检验和创新是三道不同的门

在这一轮之前，我容易把“真实问题加上干净实验”理解为足够好的研究切入点。现在我认为至少要依次通过三道门。第一是**存在门**：错误必须在目标场景中自然发生，并有分子和分母；第二是**因果门**：最小干预能够改变 first-broken edge，并排除合理替代解释；第三是**创新门**：干预变量和实验操作必须与最接近工作存在实质差异。Non-progress feedback 问题通过了前两道门，却在第三道门失败。

这个结论也让我重新理解“停止”。最早的停止是因为基础设施没有资格回答问题，后来的停止是因为 timing 效应不存在，这一次则是因为问题虽然存在、实验也可做，但答案已经不足以构成新的贡献。三种停止理由不能混为“实验失败”。前两种帮助定位证据边界，第三种保护研究时间，避免把工程改进误写为科学创新。到这里，我认为继续自动寻找第四个候选方向已经没有意义；下一步若要开展新研究，应由新的科学问题和新的证据来源重新起步，而不是继续从同一批已污染轨迹中挑选另一个失败类型。

11.9 多框架同条件复现：从运行资格走向 Hard pulse

在停止继续堆叠 RAVEN-M 模块之后，我没有把“暂时没有足够新的方法假设”理解为“不再做实验”。我把问题改成了一个更朴素的诊断：如果让 B3、M0、GUI-Owl-1.5-8B-Think 和 UI-Voyager 在同一台模拟器、同一任务实例、同一动作预算和 AndroidWorld 原生 evaluator 下运行，此前的 0 成功究竟主要来自 Qwen 能力下限、记忆设计、控制器，还是复现接口本身？这一轮不制造新的方法名，也不预设某个框架应当获胜，而是先要求每个 arm 证明自己有资格被比较。

首次 S1 暴露了一个重要边界：GUI-Owl 的图像请求在生成前被服务端拒绝，因此不能把它记作 0 分。随后我修复的是通用传输契约和 seed/实例冻结，而不是针对任务答案修改 controller；失败尝试被单独保留，修复后的任务也只作为新的匹配开发资格轮 S1b。S1b 固定 seed 为 20260806，使用完全相同的 ClockTimerEntry 与 MarkorCreateNote 实例，每项最多 12 个 Android 动作，并要求至少一次可解析决策、一次真实动作、可观察页面变化、完整 initialize-evaluator-tear-down 生命周期，以及成功模型调用覆盖实际步数。这里的“通过”只表示端到端链路有效，不表示任务成功。

表 19: 四框架匹配 S1b 资格轮。任务成功率不是资格条件；四个 arms 均完成了合法生成、动作执行与 evaluator 闭环。

Arm	Timer 任务	Markor Note 任务	S1b 资格	解释
B3	0 (4 步/5 调用)	0 (12 步/15 调用)	通过	两项均有合法 evaluator 结果
M0	0 (12 步/19 调用)	0 (12 步/21 调用)	通过	完整多角色链路运行，但未成功
GUI-Owl-1.5	0 (12 步/12 调用)	1 (9 步/9 调用)	通过	唯一成功单元，原生控制器可运行
UI-Voyager	0 (12 步/12 调用)	0 (11 步/11 调用)	通过	两项均形成合法 evaluator 结果

S1b 的结果不能被解读为 GUI-Owl 普遍最强：它只有两个开发任务和一个 seed，唯一作用是证明四套链路均已脱离接口失败，因而可以进入有界 Hard pulse。Hard pulse 预先冻结 H01 BrowserMultiply、H06 MarkorMergeNotes、H09 OsmAndTrack 和 H17 SportsTrackerActivitiesOnDate 四项任务，共 16 个 cells；每项都使用单任务 manifest、固定参数 hash、冷启动模拟器和不因模型错误重跑的规则。服务器离线前完成了 H01、H06 共 8 格，其中 7 格到达 evaluator，GUI-Owl 在 H06 中因原生 JSON 解析失败没有形成任务分数。7 个可计分单元均为 0 成功。

表 20: 截至服务器离线时的四框架 Hard pulse。括号为环境步数/模型调用；“-”表示没有合法 evaluator 分数，不能按 0 分处理。

ID	B3	M0	GUI-Owl-1.5	UI-Voyager	当前边界
H01	0 (19/26)	0 (22/34)	0 (22/22)	0 (22/22)	4/4 已评估
H06	0 (15/21)	0 (53/81)	- (21 次生成)	0 (13/13)	3 格评估，1 格控制器失败
H09	基础设施无效 (90/110)	未运行	未运行	未运行	模型隧道在第 90 步断开
H17	未运行	未运行	未运行	未运行	GPU 服务器离线

11.10 部分 Hard 结果真正告诉了我什么

H01 的共同失败说明“看到并保存数字”与“把数字正确变成答案”不是同一个能力。B3 和 GUI-Owl 都提交了 120；UI-Voyager 提交 2016；M0 的轨迹则保留了 (2, 3, 4, 5, 8) 这一组局部值，却提交 240，而正确乘积应为 960。这里至少存在 observation、retention、transformation 和 answer commitment 四个环节；即使记忆内容正确，下游计算或提交仍可能失败。因此，不能看到 memory 中出现了正确值就宣称记忆机制有效，也不能把最终错误简单归因于“模型忘了”。

H06 更接近我真正关心的 *Correct Memory, Wrong Target*, 但它与此前失败的 reminder-timing 假设不是一回事。GUI-Owl 的内部思考曾正确保留三个源文件字符串, 并在发现第一次输入错误后开始纠正, 但随后生成了无法解析的 JSON; UI-Voyager 读取了源笔记内容, 却把换行和目标文件名输入到源笔记并保存; M0 维持了合并顺序, 却用类似“某文件的内容”这样的占位表达代替真实值。换句话说, 失败不只发生在“有没有记住”, 还发生在“把哪个值写入哪个对象的哪个字段”和“能否把意图序列化成环境可执行动作”。这使我把研究对象进一步缩小为 **从证据到目标字段再到可执行动作的绑定链**, 而不是继续给 memory store 增加状态标签。

这组自然轨迹比合成 timing pilot 更有启发性, 但目前只能作为机制线索。它来自单一 H06 实例, 没有目标绑定干预、独立 seed 或 held-out App; 而且 4 个 arms 的控制器和模型都不同, 不能用一次失败直接识别 memory 的因果作用。合法表述是: 自然 Hard 任务中确实观察到“局部信息正确、落地对象或值错误”的现象, 值得在服务器恢复后完成剩余 pulse 并设计最小配对干预; 不合法的表述是: 现有 8 格已经证明了一个普遍的新方法问题。

H09 的中断进一步提醒我区分科学失败和基础设施失败。B3 已执行 90 步、110 次模型调用, 并长时间停在 Rotenboden 搜索结果页, 反复把静态搜索结果解释为“区域仍在下载”; 但在 evaluator 调用前, 本地到服务器的模型隧道断开。该轨迹可以用于分析视觉语义误读和恢复循环, 却没有合法 reward, 已被单独归档为 infrastructure-invalid, 正式槽位保持为空。当前汇总因此明确写为 16 格中 8 格完成、7 格可计分、0 成功, 而不是“8 格全失败”或“四框架 Hard 结果为 0”。

11.11 创新性与完成度边界

当前部分结果没有把工程复现自动变成方法创新, 也没有支持任何跨框架排名。它的价值在于把过去笼统的“记忆没有提升”拆成了更细的可检验断点: 值是否被保留、值是否经过正确变换、值是否绑定到正确目标字段、动作是否满足控制器语法、执行后是否由 evaluator 闭环。对夏令营报告而言, 这已经形成一条可复核的实验与思考链; 对研究论文而言, 仍需要完成 H09/H17、增加独立实例, 并用共享控制器或最小干预隔离“记忆正确但执行绑定错误”的因果效应。

11.12 对齐官方 Qwen 控制器后的完整 Hard 基线

早期 B3 与 M0 的 0/19 首先说明自建链路没有完成 Hard 任务, 却不能单独证明 Qwen3-VL-32B-Instruct 本身在这些任务上必然为 0。老师交流后, 我重新检查了这个推断中隐藏的混淆: 当模型、提示词、动作协议、历史组织方式和控制器同时改变时, 最终 0 分无法定位究竟是哪一层首先失效。为建立可解释的能力参照, 我保留 AndroidWorld 原生初始化、动作执行和 evaluator, 只把模型替换为夏令营指定的 Qwen3-VL-32B-Instruct, 并尽量对齐 Qwen 官方 Mobile Agent 的系统提示、动作空间、观察组织和历史回传方式。

该实验不是为了事后挽救 RAVEN-M，而是为了回答一个更基础的问题：在官方式控制器下，这一底座究竟能否在同一 Hard 任务域产生非零结果。

正式集合包含 19 个 Hard 任务类，每类固定 3 个 seeds，共 57 个唯一 task-seed keys。运行期间不因模型犯错重跑；传输失败、任务初始化和 evaluator 状态分别记账，只有完成完整生命周期的任务才进入分母。表21 给出冻结汇总。

表 21：对齐官方控制器后的 Qwen3-VL-32B-Instruct 完整 Hard 基线

指标	结果	含义
完整任务单元	57/57	19 类任务，每类 3 个 seeds，均有原生 evaluator
完整成功	7/57 (12.28%)	非零结果证明底座并非完全无能力
正奖励/部分奖励	9/2	2 个 Markor 跨应用任务只完成部分要求
可计入模型调用	1,175	用于反映运行成本，不把传输重试当作科学调用
错误完成声明	21 episodes	模型声称完成，但底层 evaluator 为 0
重复状态/停滞	39/14 episodes	重复较普遍，但并非所有失败的最早原因

成功并非均匀分布。SimpleCalendarAddOneEvent 在 3 个 seeds 上全部成功；ExpenseDeleteMultiple2 成功 2/3；RetroSavePlaylist 和 SportsTrackerTotalDurationForCategoryThisWeek 各成功 1/3。其余 13 类任务全部失败。这个分布比单一总成功率更有解释力：日历事件创建具有清晰入口、少量字段和直接保存闭环，因此较稳定；删除多笔支出需要识别多个对象，仍可在部分 seed 上闭环；而跨应用读取、计算、目标字段写入和多阶段地图操作更容易在中间链路失真。两个部分得分的 Markor-SMS 单元尤其说明，模型可以完成前半段的信息取得，却不能把所有要求可靠地落到最终对象上。

这次对齐也修正了我前期的研究判断。此前 B3/M0 全零中确实包含控制器与官方运行方式未充分对齐的设置性问题，不能把它们全部归因于模型能力或记忆机制；但官方式基线只有 7/57，也说明对齐接口并不会自动解决长程任务。更准确的结论是：早期系统先受到控制器能力下限限制，修正后才显露出任务级规划、目标绑定、循环恢复和完成验证等更深层问题。后续记忆实验必须建立在这个非零且可复核的底线上，否则小幅提升没有可识别空间。

为了判断失败是否仍主要来自最初的应用入口，我又对 57 条冻结轨迹做了零调用审计。对每类任务，我只按照任务文字标记第一个应进入的任务相关应用：例如先从 Gallery、Markor 或 VLC 读取材料的任务，把源应用而不是最终写入应用视为正确入口。结果见表22。49 条轨迹第一次离开桌面后就进入正确应用；6 条先误入菜谱、Chrome、SIM Toolkit、时钟或录音机，随后自行恢复；另外 2 条始终在 Launcher 上重复上滑，没有进入图库。正确首入组成功 6/49，点错后恢复组成功 1/6，未进入组为 0/2。

表 22: 官方式 57 格 Hard 轨迹的首个任务相关应用进入审计

启动类别	轨迹数	成功数	解释
第一次进入正确	49	6	入口不是这些轨迹的首要瓶颈
点错后恢复	6	1	图标定位造成动作浪费，但早期错误不必然导致失败
始终未进入	2	0	两条图库任务在桌面循环，启动层本身足以造成失败

这个结果进一步排除了一个过宽的解释：应用入口错误真实存在，却不能解释官方式基线中的 50 次失败，因为 55/57 最终都到达了正确应用，而且一个先打开时钟的日历任务仍由 evaluator 判为成功。主要瓶颈已经移动到应用内部和跨应用交接阶段，包括控件定位、对象与字段绑定、状态更新、循环恢复以及完成证据。该审计属于 post-hoc 机制测量，不是随机化干预，因而不能估计“修复入口”会带来多少净提升；它的价值是把后续分层诊断的起点从笼统的视觉失败，缩小到“绝大多数轨迹已通过应用入口，最早断点应继续向后找”。

我随后把第二层限定为任务文字明确要求从源应用转到目标应用的 9 类任务，共 27 条轨迹。这里只检查 foreground package 是否先后出现，不根据模型自述判断“已经交接”。结果形成了更清楚的漏斗：2 条没有进入源应用，9 条只进入源应用而没有到达目标应用，16 条到达目标应用。27 条完整成功仍为 0；到达目标应用的 16 条中，只有 2 条 Markor-SMS 任务得到 0.5 分，其余 14 条仍为 0。表 23 给出冻结统计。

表 23: 官方式 Hard 跨应用任务的 source-destination 漏斗

阶段	轨迹数	完整成功	正奖励	解释
未进入源应用	2	0	0	在任务起点之前失败
只到源应用	9	0	0	来源可见，但跨应用交接没有发生
已到目标应用	16	0	2	应用级到达后仍在对象、字段或闭环处失败

这使“记住了为什么还做不对”可以被拆成三个条件：来源值是否被捕获、目标应用与目标对象是否保持、值是否真正写入并由 evaluator 闭环。当前 package 审计只测量中间的应用级到达，而且可能高估真正交接，因为打开正确 App 不等于进入正确页面。即便采用这个宽松上界，11/27 也在到达目标应用前失败；另外 16/27 则证明“到达目标”依然远不足以完成任务。因此，后续结构化记忆不能只提高 record accuracy，而必须报告 source capture、destination reach、field/object binding 与最终 evaluator 四层结果。

为了继续向后定位，我进一步选取 5 类需要新增多个 Expense 或 Recipe 对象的任务，共 15 条轨迹。AndroidWorld 隐藏参数在这些实例中给出了 42 个应创建对象；我只

在事后审计读取这些参数，并检查目标应用前台时实际执行的 `type_text` 是否包含正确的 `expense name` 或 `recipe title`。匹配前统一 Unicode、大小写和标点，因此不同引号不会造成假不匹配。结果见表24：9 条到达目标应用的轨迹对应 26 个应创建对象，动作流中只有 3 个正确对象名，覆盖率为 3/26 (11.54%)，且集中在 2 条轨迹；这两条仍全部失败。

表 24：多对象新增任务的目标应用写入漏斗

最深到达阶段	轨迹数	完整成功	解释
未到目标应用	6	0	交接前失败
到达但未执行目标内文本写入	2	0	没有把来源对象转成可执行输入
已写入，但对象集合全部错误	5	0	来源识别、筛选或保持已经失真
至少写入一个正确对象名	2	0	只覆盖部分对象，字段与保存仍未闭环

这一层修正了我此前对“正确记忆、错误目标”的理解：在自然 Hard 轨迹里，很多案例还没有资格被称为“正确记忆”，因为正确对象集合根本没有进入目标动作流。两条例外也同样重要：Agent 分别输入了一个和两个正确菜谱名，却仍未完成所有对象、描述、方向与保存要求。因此结构化记忆的最小评测单位不应是一段摘要是否看起来合理，而应是 `expected object recall`、逐字段一致性、保存对象数量和最终 `evaluator` 四级闭环。只有先证明来源对象被完整、可核验地捕获，才有资格测试 `destination binding` 是否带来额外收益。

11.13 原材料覆盖诊断：高精度局部记忆仍可能系统性缺项

对象写入下界提出了一个更早的问题：正确对象没有进入目标动作流，究竟是模型打开了错误材料、没有浏览完整，还是看见后无法识别？我先把范围收窄到 9 条从 Markor 读取批量对象的轨迹。它们全部进入 Markor，8 条进入 AndroidWorld 夹具中指定文本文件的 `DocumentActivity`，因此“完全没有找到源应用”已不是主要解释。随后我冻结这 8 条轨迹中 Agent 实际看过的 13 张文档截图，用同一 Qwen3-VL-32B revision 做一次离线可见对象提取；隐藏的 `row_objects` 只在输出后用于评分，未进入模型提示。

这次诊断的门槛在调用前固定为：13/13 合法 JSON、精确率至少 80%、召回率至少 60%，且至少 4/8 条轨迹找齐全部目标。结果前两项中的格式与精确率通过：13 次输出全部合法，预测的 11 个对象全部正确，精确率为 100%、误报为 0；但 21 个目标只找回 11 个，召回率 52.38%，只有 3/8 条轨迹完整找齐，因此总资格门失败。这个结果不是一个可上线的记忆模块，却揭示了“高可信记忆”的反直觉风险：局部记录可以条条正确，集合仍然因为没有翻到全部页面而严重缺项。

把文档打开、离线可恢复对象、目标应用到达和真实文本动作连接起来后，可得到表25。8 条打开文档的轨迹中，只有 5 条从已观察画面至少恢复一个正确对象，5 条随后

到达目标应用；只有 4 条同时满足“已有正确对象且到达目标”，最终仍只有 2/9 在目标应用中实际输入过正确对象名，完整成功为 0。换言之，当前损失不是一个单点故障，而是沿 source coverage、object capture、handoff、field write 与 evaluator closure 连续累积。

表 25: 9 条 Markor 批量新增轨迹的源材料到最终写入累积漏斗

累积条件	轨迹数	机制含义
进入 Markor	9/9	源应用入口通过
打开指定文档	8/9	文件级身份与页面通过
已观察画面至少恢复一个正确对象	5/9	局部对象捕获通过
已观察画面找齐全部对象	3/9	对象集合覆盖通过
打开文档后到达目标应用	5/9	跨应用交接通过
同时有正确对象且到达目标应用	4/9	具备写入的最低前提
实际输入至少一个正确对象名	2/9	部分值进入执行流
AndroidWorld 完整成功	0/9	字段、保存与最终状态均闭环

我又直接审计了这 8 条轨迹在 `DocumentActivity` 中的规范动作，得到一个比视觉识别失败更早的过程证据：8/8 都没有执行任何向前浏览文档的纵向滑动，合计为 0 次。部分轨迹直接离开 Markor，部分声称已经筛选完记录，另有一条甚至在没有遍历全文时回答“没有符合条件的菜谱”。因此 52.38% 的召回不能简单解释成 OCR 或视觉模型看不懂文字；当前控制器首先缺少的是“怎样证明自己已经读完长材料”的覆盖协议。不同截图 hash 可能来自选中文本、光标或页面局部变化，也不能替代遍历证据。

这一步改变了我对结构化记忆实验顺序的判断。下一轮不能直接把现有摘要改成 JSON 后比较最终成功率，因为结构化表示只会更整齐地保存一个不完整集合。更可识别的顺序应是：先给浏览行为定义可核验的 coverage contract，证明目标记录已被逐页覆盖；再测 object-level recall；只在召回充分的轨迹上测试 destination binding；最后分别检查逐字段写入、保存对象数和 evaluator。当前结果未授权在线结构化写入实验，也不允许在同一 13 张截图上反复改提示词越过门槛。

11.14 可执行覆盖门：滚动增加了，但完整信息没有随之出现

提示词式覆盖要求在第一条开发任务上仍以 0 次向前滚动离开文档，因此我没有继续修改同一段文字，而是在新 seed 20260809 上冻结三条 Markor 跨应用任务，比较官方方式基线与外部可执行覆盖门。干预组在 `DocumentActivity` 内阻止 Home、Back、answer、terminate 和其他非向前扫描动作，并把它们替换为固定纵向滑动；两组都只运行到首次离开源文档或 60 步上限，不使用最终 evaluator 形成任务成功率。随后，冻结提取器对

Agent 实际观察到的每张去重文档截图各调用一次，隐藏 `row_objects` 只在输出后用于评分。

表 26: seed 20260809 的同实例源阶段覆盖对照。资格门要求干预组八项全部通过。

指标	基线	覆盖门	判定
打开指定文档	1/3	2/3	未达到 3/3
至少一次向前滚动	0/3	2/3（共 64 次）	未达到 3/3
页尾证明	0/3	1/3	未达到 3/3
提取器合法输出	3/3	65/65	通过
对象微平均精确率	0（无正预测）	100%（1 TP、0 FP）	通过
对象微平均召回率	0/8	1/8（12.5%）	未达到 75%
召回率绝对增量	—	12.5 个百分点	未达到 20 个百分点
完整找齐对象的任务	0/3	0/3	未达到 2/3

覆盖门确实改变了执行，而不只是改变模型自述。支出任务中，模型 7 次提出离开或非扫描动作，控制器均将其改写为向前滑动；整条轨迹共滚动 53 次，但 60 步内仍未证明读到底，也没有恢复出 `Pet Supplies` 或 `Dividends`。另一条菜谱任务已经进入 `Markor` 文件列表并看见 `recipes.txt`，却没有打开文件便返回桌面；由于门只在 `DocumentActivity` 生效，它在错误发生后才有权限介入。第三条菜谱任务打开了文档并滚动 11 次，但只恢复出一个正确对象，另外两个仍缺失。零模型调用的冻结轨迹审计进一步发现：该任务第 12 步首次宣称页尾，但第 14 至 19 步的同方向滑动均产生约 16.8%–18.5% 的页面像素变化，连续六次后续证据直接推翻了页尾判断。总体上，调用数由 82 增至 141，prompt tokens 由 323,495 增至 612,785；额外 59 次调用和 289,290 个 prompt tokens 只换来一个正确对象，没有产生任何完整对象集合。

更重要的是，唯一一次“页尾证明”被后续轨迹反驳。step 12 的滑动满足预注册判据：像素变化低于 0.001、Activity 不变且 UI-tree hash 不变，于是系统关闭 coverage；但后续同方向滑动仍产生新的文档画面。这说明一次 no-effect 既可能表示真正到达页尾，也可能只是手势未推进、编辑器焦点或渲染状态造成的暂时无变化。由此，coverage contract 不能只包含“继续滑动直到一次无变化”，而应进一步拆成：正确文件已经打开、起点得到证明、位置单调向前、终点经过连续独立确认。本轮八项资格门只有合法输出和精确率两项通过，因此不授权 destination-write 或结构化记忆实验，也不在这六个已观察 cells 上修改阈值后重跑。

11.15 最小 L4 救援：一次有价值的负结果

57 个任务中有 39 个出现重复状态，因此我没有立刻实现一个大型新框架，而是先冻结最小问题：如果动作执行后页面活动、UI hash 和像素变化都没有改变，历史是否仍应把模型原先声称的语义效果写成已经发生？干预仅在上述三项证据共同表明无变化时，把历史改写为“规范动作已经执行，但未观察到页面转换，语义效果未经验证；不要在同一状态重复相同动作”。它不修改截图、任务、模型 revision、动作预算或 evaluator，也不在看到结果后调整阈值。

表 27: L4 动作后语义确认的 matched diagnostic。两组任务均为同一 task-seed、同一模型和同一预算。

任务	条件	Reward	模型调用	最长停滞段	终止原因
Expense 删除任务	原始	0	34	27	达到步数上限
Expense 删除任务	L4 干预	0	34	23	达到步数上限
OsmAndTrack	原始	0	120	72	达到步数上限
OsmAndTrack	L4 干预	0	49	2	错误声称成功后提前结束

结果没有通过预注册资格门。Expense 的最长停滞段只下降 14.8%，低于预设的 50%，且最终仍失败；OsmAnd 虽然把最长停滞从 72 步降到 2 步，却不是因为任务变好，而是模型更早错误声明完成。回放显示，模型先后通过普通搜索访问 Schaan、Balzers、Planken 和 Malbun，这些动作都造成了真实页面变化，因而不会触发 L4 无变化规则；进入新建路线页面后，模型却把此前“访问过地点”当成“已经加入路线的途经点”，点击完成并提前退出。也就是说，L4 能阻止部分虚假的动作历史，却不能识别一个产生真实页面变化的错误动作。

这个负结果使我的问题进一步前移：重复动作是可见症状，但最早断点有时发生在对象角色判断。普通搜索页能够证明 location_visited，却不能证明 waypoint_added；只有轨迹编辑器中的点数或列表发生预期变化，才有资格更新“途经点已加入”这一子目标。下一步如果继续实验，应先建立这种任务无关的对象角色和状态变更资格，而不是继续增加更多循环提示。这里我只把它写成由失败轨迹导出的待检验假设，不把尚未实现的机制包装成正向创新。

11.16 对象角色证据：自然语言规则仍然不够

L4 的失败提出了一个更窄的问题：即使页面确实发生变化，模型能否只把它当作当前截图真正证明的事实，而不把“访问地点”写成“加入途经点”、把“保存收藏”写成“创建地图标记”？在查阅 TSR、VeriGUI、StepReflect 和 VeriSafe 等相邻工作后，我没有把通用状态跟踪或动作后验证重新包装成创新；我先对 57 格轨迹做零调用回顾，找到跨 5 类任务、4 个应用族的 13 个干净对象角色错配，再冻结 3 个同实例诊断。干预只在官方

提示词后追加一条通用规则：动作先记为尝试，只有当前截图直接证明对象类型、父级、字段、容器和操作都匹配，才允许更新任务进度。

表 28: 对象角色证据提示的同实例诊断。干预前后模型、seed、任务参数、原生预算与 evaluator 均相同。

任务	条件	Reward	调用数	动作数	终止原因
菜谱多项条件删除	官方基线	0	15	14	错误声称成功
菜谱多项条件删除	证据规则	0	25	24	错误声称成功
收据图片复制	官方基线	0	10	9	错误声称成功
收据图片复制	证据规则	0	16	16	达到原生步数上限
OsmAnd 地图标记	官方基线	0	11	10	错误声称成功
OsmAnd 地图标记	证据规则	0	8	7	官方输出协议无效

结果再次没有通过预注册门：3 个任务仍为 0 成功，调用数由 36 增至 49，动作数由 33 增至 47，总耗时由 718.7 秒增至 1554.3 秒。菜谱任务最能直接否定当前干预：模型实际打开的是 Android 分享面板，页面明确显示 Copy、Nearby、OsmAnd、Tasks、VLC 和 Markor，却在下一步声称“菜谱已删除”并结束。图片任务虽然没有错误宣布成功，但只是更早把 Maps 误认成图库，随后在应用抽屉反复滑动直至耗尽预算；OsmAnd 任务把 Retro Music 误认成地图，恢复后又输出官方协议不接受的 Recent 按键，解析器按设计拒绝执行。换言之，假成功从 3 次降到 1 次并非任务能力提高，而是另外两次分别变成了更早停滞和协议失败。

这一步使我的分层判断更加具体：“证据只做它有资格做的事”是正确的分析原则，却不能靠一句提示词变成可靠能力。模型在到达对象角色判断前，可能已经在应用图标定位、控件定位和循环恢复上失败；即使到达，自己的动作历史仍可能压过屏幕中的反证。因此下一步不应继续加长提示词，而应把 app/control grounding、页面转换、typed postcondition、任务状态更新和外部 evaluator 分开记录和干预，并在同一冻结轨迹上比较每层到底救回多少错误。

11.17 把 Critic 单独拿出来：终态截图仍不足以证明完成

为了区分“模型看不出反证”和“模型边操作边记历史时没有使用反证”，我又把 Critic 从动作链中完全分离。数据不再挑案例，而是机械纳入 57 格中全部 27 次显式成功声明：6 次被 evaluator 确认，21 次是假成功。独立 Critic 只看到任务文字和宣告成功前的最后一张截图，看不到动作历史、UI tree、数据库、reward 和原 Agent 的成功理由；每例固定一次调用，不作修复。这样测到的不是新框架成绩，而是“单张终态图是否足以成为完成证据”。

表 29: 截图式完成审查的冻结结果。普通准确率受 21:6 类别不平衡影响，因此以两类召回率和平衡准确率为主。

指标	结果	预注册门槛	判定
格式有效	27/27	27/27	通过
假成功拒绝率	19/21 (90.48%)	至少 16/21	通过
真成功放行率	2/6 (33.33%)	至少 4/6	未通过
平衡准确率	61.90%	至少 70%	未通过

77.78% 的普通准确率看似不低，却恰好等于“27 条全部拒绝”的笨规则，因此不能包装成有效分类器。Critic 做对的部分很清楚：它能指出短信成功不代表 Markor 笔记成功、复制成功 toast 不证明目标是 Download、地图出现点也不证明对象类型和坐标准确。但 4 个真成功被拒绝也不是简单的模型保守：删除属于负事实，当前列表没显示对象不能证明数据库已删除；导出属于屏幕外副作用，播放列表详情不能证明文件已进入 Downloads；同一个日历任务是否被放行，还取决于结束时有没有主动停在包含完整字段的详情页。

两个假成功反而被 Critic 放行更有启发性：截图显示搜索“lettuce”或“almond milk”后“No recipes found”，Critic 就推断目标菜谱已全部删除。问题是截图没有证明搜索覆盖的是 directions 字段；“这个搜索算子返回 0 项”不能推出“evaluator 的数据库谓词为 0 项”。所以真正缺少的不是更严格的口吻，而是证据来源的作用域和权限。下一步完成验证应先按正事实、负事实、多对象、目标目录和跨应用副作用分类，再决定使用详情页、穷尽列表、文件系统或数据库证据；不应把一张终态图当作所有任务的统一裁判。

12 有效性威胁

第一，Hard breadth 处于 capability floor，无法稳定估计小效应；15 个缺少 answer action 的 cells 又引入了接口性零分。本文同时报告 95-cell 全量结果和 80-cell 受支持结果，但不能据此得出 memory 普遍无效。

第二，r39/r56 仅包含四类配对任务，且 controller 在相同任务上经历连续诊断，可能逐渐形成域内 task-pattern engineering。它们适合工程 qualification，不适合作为未见泛化证据。

第三，H17 在 r61–r78 中被反复观察，r77 成功存在 development contamination，且 prompt 超过预注册 context cap。本文保留该案例用于机制解释，不将其纳入 efficacy claim。

第四，自然 contradiction 审计不是双人盲法，15 个 flags 中 14 个可能只是兼容改述。未来需要固定 annotation guide、双人独立标注并报告一致性。

第五，公开仓库默认不包含模型权重、完整 runs、results 和大量截图，虽可核查代码、protocol、报告与 hash，但不能仅凭 GitHub 重新计算全部 trajectory statistics。正

式投稿需要发布压缩事件、evaluator outputs、标注和 artifact manifest。

第六，v0.2.2 的严格 pixel-hash 稳定条件会受到 launcher 动画、状态栏和其他非任务像素变化影响；而 package/all 也不能覆盖所有纯视觉状态。当前结果只能说明 outcome oracle 存在 false rejection 风险，不能事后把第三格重标为成功。测量契约开发阶段观察过的 Settings、Camera 和通知栏轨迹也已经污染，未来校准必须换用独立 trace 集和全新 live probes。

第七，v0.2.3 暴露了轨迹采集生命周期本身的有效性威胁：动作前证据可能在 App 已打开但 all 尚未就绪时被过早采样，清理阶段的次要异常也可能阻止完整账本落盘。9 条 DEV replay 已被开发过程观察，不能替代独立 held-out；本轮 0 条正式轨迹意味着 action-conditioned oracle 仍只有实现与机制证据，没有外部效度结论。

第八，Correct Memory, Wrong Target 的 96 个条件单元全部用于开发期机制筛查，来自 8 个模板、3 个 App 场景和单一 Qwen revision；实验没有 live Android action，也没有新鲜 held-out corpus。三轮之间的目标标签条件是配置级变化而非同一冻结实验内的完整随机化，因此结果足以停止当前方法开发，却不能形成跨模型、跨任务的普遍零效应定理。

第九，GPT Pro 参与了方向规划和最终反向审查，可能引入新的表述与选择偏差。本文通过提供完整仓库材料、保留原始失败、冻结预注册门槛并由本地 artifact 复核关键数字降低这一风险；但 AI 审阅不能替代同行评议，也不构成创新性证明。最终主张只以可检查的代码、原始输出、实验契约和公开文献为依据。

第十，循环发生率来自冻结 forensic report，而不是本次重新对全部 raw episodes 进行双人独立标注。80 个 supported cells 实际由 16 个任务实例乘 5 种设置构成，并非 80 个独立任务；14/44 又是嵌套在 19 个 M0 episodes 内的事件，且全部 Hard 实验只有一个 seed。因此，表18 只描述当前 suite 的工程失败分布。即使本地 raw 完全确认这些计数，也只能通过存在门，不能恢复已经失败的创新门。

第十一，修复初始 S1 的图像传输和实例错位后，S1b 虽使四个 arms 在相同两个 non-Hard 任务、同一 seed 和冻结实例上全部通过资格门，但它仍是开发期端到端资格筛查，而不是效能估计。GUI-Owl 的 1 个成功只能说明该链路能够闭环，不能据此进行模型排名或声称普遍优势。

第十二，四框架 Hard pulse 在当时只完成 16 格中的 8 格，其中 7 格具有合法 evaluator 分数且均为 0；H06 的 GUI-Owl 属于控制器解析失败，H09 的 B3 又在 evaluator 前遭遇模型隧道中断，均不能伪装成普通任务 0 分。后续完成的 57 格官方 Qwen 基线解决的是同一底座在官方式控制器下能否形成非零能力参照，并没有补齐四框架剩余槽位。因此，本文可以比较早期全零与官方式基线的设置边界，但仍不能对四个异构框架作性能排名。

第十三，官方式完整基线虽然包含 3 个 seeds，但所有任务使用同一模型 revision、固定解码设置和同一 AndroidWorld 环境，部分 seeds 的初始可见状态可能高度相似，不能把 57 格当作 57 个完全独立样本。L4 matched diagnostic 又是在已观察失败分布后选择的两个机制样本，适合做因果排错，不是未见测试集上的方法效能检验。其 0 成功只能否定当前干预的充分性，不能否定动作后验证的一般价值。

第十四，对象角色证据的 3 个同实例案例同样来自自己观察过的开发集，而且提示规则改变了模型较早的图标选择和恢复路径，使其中 2 个任务没有到达原基线的目标错配位置。因此本轮只能严格否定“单条自然语言规则足以改善任务”；它不能估计一个结构化、外部执行的 typed-evidence 模块的效应，也不能把假成功数量下降解释为能力提升。

第十五，截图式完成 Critic 的 27 条记录来自同一 57 格开发集，类别比例为 21 个假成功对 6 个真成功；普通准确率会被 reject-all 策略夸大。更重要的是，evaluator success 与 screenshot-provable success 不是同一标签：删除、导出和跨应用状态可以真实完成却无法由终态截图单独证明。因此本轮适合否定统一截图裁判的充分性，不能作为通用视觉 verifier 的训练集或性能排名。

第十六，首个任务相关应用审计依赖按任务文字人工冻结的 package 对照，并且把第一次进入 package 作为入口成功；它不检查应用内页面是否正确，也不能证明入口修复对最终 reward 的因果效应。49 条首次正确、6 条恢复和 2 条未进入只能用于定位故障发生层次，不能视为一个新的模型准确率指标。

第十七，跨应用漏斗同样以 foreground package 作为宽松的 reach 上界。目标 package 出现不证明来源值被正确读出、页面角色正确或字段写入完成；因此 16/27 不是跨应用信息传递成功率。该审计只能确认 11 条在应用级交接前已经失败，并把另外 16 条的断点范围缩小到目标应用内部或最终验证。

第十八，对象标识符审计只检查目标应用中的 type_text，是可执行写入的严格下界；它可能漏掉预填字段、非文本选择或其他输入通道，也没有核对金额、描述、方向与保存状态。3/26 只能说明正确名称进入了动作流，不能证明完整对象被正确创建；相反，未匹配也应解释为“动作日志中未见规范化正确标识符”，而不是直接证明模型内部从未感知该对象。

第十九，Markor 可见对象提取仍使用已观察过的开发轨迹，8 条轨迹和 21 个对象都不足以估计一般化性能；13 张截图也不是主动覆盖全文后取得的独立样本。100% 精确率与 52.38% 召回率只能说明当前冻结提示在这些已见页面上偏保守，不能证明 Qwen 对长文档提取普遍高精度。Markor DocumentActivity 判据依赖 AndroidWorld 为该任务写入指定文件的固定夹具，且“进入文档”不等于“浏览全文”。本轮资格门失败后没有在同一批截图上调参重跑，因此它是机制边界而不是新方法成绩。

第二十，文档覆盖审计观察到 8/8 轨迹均无向前纵向滑动，但这仍是过程相关而不是随机化因果估计。目标可能恰好位于首屏，长按全选也可能把屏幕外文本送入系统剪贴板；反过来，执行一次滑动也不等于已覆盖全文。因此 0 次滑动足以否定“原 Agent 已按可审计协议遍历文档”，却不能单独证明增加滑动必然提高最终成功率。需要在新实例上冻结终止条件、页面去重和调用预算后再做 matched coverage pilot。

第二十一，可执行覆盖门的三任务 matched pilot 仍是开发集机制实验，而且干预组同时改变了 coverage prompt 与外部动作门，不能分离二者的单因素效应。干预把向前滚动从 0 次提高到 64 次，却只把对象召回从 0 提高到 12.5%，没有通过预注册门；该结果只能否定当前实现的充分性。更具体地说，门只在 DocumentActivity 激活，无法阻止模型在文件列表阶段离开；可编辑长文档中的滚动又可能耗尽 60 步。一次低像素变化且 UI

hash 不变的滑动还被后续新画面证明是假页尾，因此当前 `bottom_attested` 不能解释为真实全文覆盖。操作上，基线提取发生在干预 suite 前，而预注册写的是两组 suite 完成后再提取；隐藏标签没有进入在线 Agent，提取器也不共享会话状态，但该顺序偏差仍被保留，不能省略。

13 结论

本文完成了夏令营课题要求的完整工程与实验链路：调研 GUI Agent memory，部署 Qwen3-VL-32B-Instruct AndroidWorld baseline，实现多角色 RAVEN-M，完成 Hard task 系统评测、性能与成本比较、案例和误差分析，并整理代码、脚本、配置与复现材料。早期 Hard breadth 中 M0 与 B3 均为 0/19，而 M0 增加约 33% 模型调用、53% tokens 和 52% wall time；r56 中 M0 由 B3 的 4/4 降至 3/4，同时 prompt tokens 增加 69.53%。这些结果没有证明复杂记忆优于简单摘要，也没有证明长期 lifecycle 是当前 AndroidWorld 的主要瓶颈，但它们也不能脱离控制器设置直接代表 Qwen 的官方式能力。

项目随后经历了三次重要收缩。第一次把宽泛的“可靠记忆”拆成 Creation-Binding-Retrieval-Authorization-Execution-Verification 价值链，并用 EEST-AC 最小实验定位动作契约、测量与采集资格问题。第二次把实体绑定进一步缩小为 Correct Memory, Wrong Target timing 假设；96 个开发期条件单元和 192 次调用中，正确值回忆始终为 100%，但 Timing×Ambiguity 交互始终为 0，因此效应门失败。第三次回到自然失败分布，发现 32/80 个 supported cells 至少连续重复同一动作 10 次，14/44 个 loop feedback events 立即重复原动作，并把 first-broken edge 定位为 recovery feedback 到 next-action policy update。

第三个问题通过了存在门和因果门，却没有通过创新门。VLAA-GUI 已经覆盖重复无效动作或状态触发、强制交互方式或策略切换、反思信号绑定以及闭环任务结局；移动端 action-effect verification 和 backtracking 也已有相邻工作。因此，AndroidWorld、Qwen 和本地 controller 的差异只能支持工程迁移或复现，不能支持新的方法贡献。这里我仍保留 **STOP_RESEARCH_METHOD**：不实现 Destination-First Binding Gate 或 anti-loop hard binding。但后续的多框架计划使 **NO_GO_NEW_EXPERIMENT** 被更精确的边界取代——不再为旧假设增加方法实验，允许执行预注册、同条件的强框架复现，用数据判断此前 0 成功究竟位于模型、控制器还是接口层。

初始 S1 首先暴露了 GUI-Owl 图像传输失败和跨 runner 实例错位。完成通用接口与 seed 冻结修复后，匹配 S1b 中四个 arms 均通过端到端资格门；后续部分 Hard pulse 又观察到计算变换、目标字段绑定和动作序列化断点。更关键的是，重新对齐官方 Qwen Mobile Agent 运行方式后，19 类任务、3 个 seeds 的完整基线取得 7/57 成功和 2 个部分得分，证明早期全零同时混入了控制器设置下限，而不是模型在 Hard 域绝对无能力。与此同时，21 次错误完成声明、39 个重复状态 episode 和 13 类任务全失败也表明，官方式控制器只是建立了可用底线，并未解决长程闭环。

在这个底线上，L4 matched diagnostic 给出了进一步的负证据：用真实页面变化约束

历史语义，可以减少部分无变化循环，却没有带来任务成功；OsmAnd 甚至在停滞减少后更早错误结束。最早错误并不是“动作没生效却被写成生效”，而是普通地点搜索产生了真实页面变化，却被错误解释为向路线加入途经点。由此，我把下一步问题从循环抑制推进到对象角色资格：记忆不仅要保存发生了什么，还必须说明这个证据能够证明哪个子目标，不能证明哪个子目标。

随后完成的 3 任务对象角色匹配诊断又排除了一个看似直接的解法：仅在提示词中要求精确核对对象、容器和操作，仍为 0/3 成功，调用数从 36 增至 49。菜谱案例甚至在分享面板上声称删除成功，另外两例则更早失败于应用定位、重复滑动和协议格式。这说明对象角色资格是必要的分析语言，却尚未成为有效机制；真正可检验的下一步应把视觉 grounding、转换检测、typed postcondition、任务状态更新和最终 evaluator 分层，而不是继续让同一个模型用更长文字同时承担所有职责。

把 Critic 单独拿出来以后，结论又收紧了一步：27 次成功声明的截图式复核能够拒绝 19/21 个假成功，却只放行 2/6 个真成功，平衡准确率 61.90%，没有通过门槛。它既会因为删除或导出的关键状态不可见而拒绝真实完成，也会把“搜索无结果”越权解释为“数据库目标已清空”。因此下一步不再是统一加一个 Critic，而是先给不同任务谓词指定可证明它的证据通道，并让控制器在结束前主动构造可验证视图；对视觉不可证明的负事实和外部副作用，则必须使用 evaluator、文件系统或数据库状态。

与此同时，对 57 条轨迹的应用入口审计表明，49 条第一次进入正确、6 条点错后恢复，只有 2 条始终没有进入任务相关应用。它说明视觉图标定位仍会浪费预算，却不能解释绝大多数失败：55 条轨迹最终已经到达正确应用，后续断点才是主要研究对象。这也使“逐层诊断”从一个计划变成了已有测量原则：先确认入口层是否通过，再分析应用内动作、对象绑定、跨应用副作用和完成证据，避免把所有 0 分重新归结为底座视觉能力不足。

进一步抽取的 27 条跨应用轨迹又把后续断点分成两道漏斗：11 条没有到达目标应用，16 条虽已到达目标应用却仍无完整成功，只有 2 条取得部分奖励。这直接说明“记忆中已有来源信息”与“任务完成”之间还隔着目标保持、字段绑定、写入动作和 evaluator 闭环。下一阶段的结构化记忆实验必须分别报告这四层，而不能再用一条 memory accuracy 或最终 reward 掩盖中间发生了什么。

在 15 条多对象新增轨迹中，这个漏斗还能再向后分：9 条到达目标应用，但 26 个应创建对象只有 3 个正确名称进入实际文本动作，且仍没有完整成功。这说明当前自然失败并不普遍符合“正确记忆已经形成，只是绑定错目标”；更常见的是来源对象捕获、筛选、全量覆盖和目标写入同时不完整。新的记忆方法若要有可识别贡献，必须先在 object-level recall 上建立可靠正例，再对同一批已正确捕获的对象测试 destination binding，而不能把前端感知错误混入记忆效应。

Markor 源材料诊断进一步解释了这种缺失从何而来：9 条轨迹全部进入 Markor、8 条打开文档，但冻结提取器在真实已观察截图上只有 52.38% 召回和 3/8 完整覆盖，未通过预注册门；沿完整漏斗最终只有 2/9 输入过正确对象名，仍为 0/9 成功。对我而言，这比简单归因于“记忆没用”更具体：记忆内容即使没有一条是假话，也可能因为文档

没有被系统遍历而缺少近一半目标。所以下一步的第一个实验对象应是可证明的 source coverage，而不是直接增加 memory 长度或把自然语言摘要改成结构化字段。

进一步的动作审计发现，8 条已打开文档的轨迹没有一次向前纵向滑动。这个事实把下一步从抽象的“提高召回”收缩成了可执行问题：让 Agent 在离开源文档前提交页面覆盖证据，并把终止条件绑定到“继续滚动已不能产生新页面”，而不是绑定到模型一句“我已经读完”。只有这个 coverage gate 在新实例上先提高 object recall，结构化记忆与目标绑定实验才有干净的正例基础。

新的 matched coverage pilot 又把这个判断推进了一层：外部动作门能够真实拦截过早退出，并把 0 次向前滚动变成 64 次，但三条任务只有两条打开文档、一条得到形式上的页尾证明，召回率也只从 0 提高到 12.5%。更关键的是，形式页尾后来仍出现新画面，说明“动作没有造成变化”不能自动推出“文档已经到底”。因此下一步不应继续给现有门加更多提示，而应在全新实例上分别证明四个条件：指定文件已打开、扫描从可验证起点开始、位置证据单调前进、终点由连续独立的无新内容观察确认。当前结果明确停止 destination-write 和结构化 memory 扩展。

对我而言，项目最终形成的研究能力比某一次正向数字更重要。我学会了区分模块激活与任务效用、动作发生与任务完成、开发诊断与 held-out 证据，也进一步认识到“问题真实”“实验可做”和“贡献新颖”是三种不同判断。真正的科学进展既包括承认早期实验存在设置性混淆，也包括修正后建立非零基线，再用最小干预排除一个看似合理但不充分的解释。截至 2026 年 8 月 8 日，旧 RAVEN-M 方法线没有取得正向净收益，四框架排名仍未完成，L4、对象角色提示和当前覆盖门三条救援均未通过资格门；但官方式 57 格基线、自然失败漏斗和三轮匹配负实验已经把问题从“系统为什么全是 0”推进为“长程任务在哪一层首先失去目标对象和完成证据，以及什么过程证据才有资格关闭这一层”。这条边界清楚的负结果和可逐层测量的下一步问题，是本阶段最可信的研究产出，而不是已经成立的方法贡献。

A 代码、配置与复现材料

公开仓库为 <https://github.com/ScottBlizzard/RAVEN-M>。夏令营核心交付与外部审阅材料对应远端 main 的冻结 commit：

0173df45e64b5bca7da782d7c9ff6e8cf0f16840

EEST-AC 后续诊断、Correct Memory, Wrong Target 三轮筛查与最终判定已推送至远端 protocol-v2-exploratory 分支。本报告更新时该公开分支对应提交为：

cfd2bbf9f80905fbd92c8ddc4e9759d0932d2076

上述分支保留了连续证据链：ecaddba 固定四框架 S0 的源码、模型、环境与零调用资格边界，a055ec0 固定初始 S1 的资格汇总、跨 runner 实例错位和 GUI-Owl 传输失败，

076f8b6 固定匹配 S1b 的四框架资格通过与 Hard pulse 授权，8088974 补齐 seed 生成冻结 Hard 实例的运行支持，cfd2bbf 加入 Hard pulse 冻结证据账本和机械汇总脚本。原始大体积运行日志保留在本地输出目录，公开分支提交用于复核协议与阶段证据的归一化材料；正式报告位于夏令营交付包，不把版式文件混入实验仓库。工作区中受保护的 r79 WIP 修改和未提交基础设施文件没有被纳入上述提交，也没有被用于改写任何旧 frozen 结果。2026_08_05_17_36.md 给出的停止结论只约束继续扩展旧方法模块；本轮执行的是有界、预注册的强框架复现。服务器离线时的未完成槽位保持为空，H09 基础设施无效尝试另行归档。关键目录与材料见表30。

表 30: 主要代码与实验材料

路径	内容
04_protocols/environment_lock.yaml	模型、Android 和协议环境锁
04_protocols/experiment_protocol.md	冻结实验协议
05_project/src/raven_m/models/	模型服务与 transformers client
05_project/src/raven_m/env/	AndroidWorld adapter
05_project/src/raven_m/controller/	episode controller 与 protocol guard
05_project/src/raven_m/history/	B0-B3 历史策略
05_project/src/raven_m/memory/	MemoryItem、manager、retrieval 与 store
05_project/src/raven_m/roles/	Planner、Executor、Memory Manager、Critic
05_project / scripts / run_frozen_hard_suite.py	Hard breadth runner
reports/breadth_gate_review_2026-07-26.md	95-cell 主结果
reports/protocol_v2_2_r56_gate_e_final.md	r56 配对结果
reports / eest_ac / eest_ac_v0_2_2_qualification_final_report.md	v0.2.2 完整决策封装资格结果
reports / eest_ac / eest_ac_v0_2_3_collection_floor_verdict.md	v0.2.3 动作条件 oracle 与采集失败判定
05_project / artifacts / role_binding_timing/	三轮 timing pilot 的冻结配置、summary 与逐单元输出
reports / role_binding_timing / CORRECT_MEMORY_WRONG_TARGET_DEV_PILOT_VERDICT_2026-08-05.md	96-cell、192-call 开发期筛查最终判定
reports/research_direction/GPTPRO_FULL_CONTROL_HANDOFF_2026-08-05.md	向独立规划审阅提供的完整证据交接
2026_08_05_17_36.md	自然循环错误筛查、最接近工作审计与 no-go 决策
05_project/metadata/multi_framework_s0_v0_2/final/	四框架零调用、零动作的 S0 资格证据与封印
05_project/metadata/multi_framework_s1_v0_2/final/	S1 资格判定、实例错位审计与 GUI-Owl 无有效分数证据
05_project / scripts / finalize_multi_framework_s1_v0_2.py	从冻结运行输出机械汇总 S1 资格门的脚本
05_project / metadata / multi_framework_s1b_v0_3/final/	匹配 S1b 的四框架资格结果与 Hard pulse 授权证据
05_project/configs/task_manifests/hard_pulse_v0_3/	H01、H06、H09、H17 的冻结任务实例与参数 hash

表 30 (续)

路径	内容
05_project / scripts / finalize_multi_framework_hard_pulse_v0_3.py	区分可计分、控制器失败与基础设施无效的 Hard pulse 汇总脚本
reports / official_qwen32b_full_hard_combined_corrected_final.json	官方式 Qwen 完整 57-key Hard 基线机器可读汇总
reports / official_qwen32b_full_hard_combined_corrected_final.md	完整基线的人类可读结果与任务级统计
reports / official_qwen32b_cross_seed_mechanism_notes_2026-08-08.md	三 seeds 机制复核、清洁重跑与失败类型记录
05_project / scripts / audit_app_launch_grounding.py	首个任务相关应用进入层的零调用审计脚本
reports / official_qwen32b_app_launch_grounding_audit_2026-08-08.json	57 条逐轨迹启动类别、事件路径与 SHA-256
05_project / scripts / audit_cross_app_handoff.py	跨应用 source-destination 漏斗的零调用审计脚本
reports / official_qwen32b_cross_app_handoff_audit_2026-08-08.json	27 条跨应用轨迹的阶段分类、事件路径与 SHA-256
05_project / scripts / audit_expected_object_transfer.py	隐藏 ground truth 对象标识符与目标写入动作的零调用审计脚本
reports / official_qwen32b_expected_object_transfer_audit_2026-08-08.json	15 条多对象新增轨迹的逐对象匹配与阶段漏斗
05_project / docs / VISIBLE_OBJECT_EXTRACTOR_MARKOR_PREREG_2026-08-08.md	Markor 已观察页面对象提取的冻结问题、模型与资格门
reports / visible_object_extractor_markor_diagnostic_2026-08-08.md	13 张真实截图、21 个对象的精确率、召回率与停止判定
05_project / scripts / audit_markor_source_funnel.py	指定文档、对象恢复、目标到达与实际写入的确定性漏斗脚本
reports / official_qwen32b_markor_source_funnel_audit_2026-08-08.json	9 条 Markor 轨迹的逐层证据、首个失败位置与事件 hash
05_project / scripts / audit_markor_document_coverage.py	文档纵向浏览动作、唯一画面与对象召回的零调用审计脚本
reports / official_qwen32b_markor_document_coverage_audit_2026-08-08.json	8 条已打开文档轨迹的 0 次向前滑动与逐轨迹召回证据
05_project / docs / SOURCE_DOCUMENT_COVERAGE_GATE_MATCHED_PREREG_2026-08-08.md	新 seed 三任务源阶段对照、资格门、停止规则与实现 hash
05_project / src / raven_m / official_qwen_mobile / source_document_coverage_gate.py	离开源文档前强制向前扫描的外部可执行覆盖门
reports / source_document_coverage_gate_matched_2026-08-08.json	0 对 64 次滚动、对象召回与八项资格判定的机械汇总
reports / source_document_coverage_gate_contract_audit_2026-08-08.json	页尾判断被后续六次同向页面变化推翻的零调用冻结轨迹审计
reports / source_document_coverage_gate_matched_2026-08-08.md	覆盖门 matched pilot 的逐轨迹解释、成本与协议偏差
05_project / docs / L4_TRANSITION_ATTESTATION_MATCHED_DIAGNOSTIC_PREREG_2026-08-08.md	L4 动作后语义确认的冻结预注册与实现 hash
reports / l4_transition_attestation_matched_diagnostic_2026-08-08.md	L4 matched diagnostic 结果、因果回放与停止判定
reports / g6_inspectable_memory.md	memory 可审计性测试

表 30（续）

路径	内容
reports/reproduction_guide.md	完整复现顺序

对象角色证据诊断的补充材料包括冻结预注册 05_project/docs/EVIDENCE_QUALIFIED_PROGRESS_MATCHED_DIAGNOSTIC_PREREG_2026-08-08.md 与完整结果 reports/evidence_qualified_progress_matched_diagnostic_2026-08-08.md；两者分别保留提示词、资格门、实现 hash，以及 3 任务匹配回放与停止判定。

截图式完成 Critic 的冻结方案与结果分别位于 05_project/docs/OFFLINE_COMPLETION_VERIFIER_PREREG_2026-08-08.md 和 reports/offline_completion_verifier_diagnostic_2026-08-08.md；27 条逐例输出保留在 runs/completion_verifier/completion_verifier_20260808T183051_50c0413f/。

基础复现顺序为：核对冻结 commit 与环境锁，检查 exact model revision，启动模型服务与 emulator，执行 readiness 和 zero-call preflight，冻结 source/config/prompt hash，运行 suite，使用 native evaluator 评分，最后 aggregate、audit 并冻结报告。

B 考核要求对应关系

表 31：夏令营考核要求完成情况

要求	状态	本文位置
GUI Agent 记忆机制调研与分类	完成	第 2 节
AndroidWorld 与 Qwen3-VL-32B baseline	完成	第 4 节
Planner/Executor/Memory Manager/Critic 框架	完成	第 5 节
保存历史、状态、变量和失败经验	完成	第 5 节
AndroidWorld Hard tasks 系统评测	完成	第 6–7 节
成功率、性能变化与资源成本	完成	第 7 节
典型成功和失败案例	完成	第 8 节
误差分析、候选假设检验与终止判断	完成	第 8–11 节
代码、脚本、配置和复现说明	完成	附录 A
Medium task 扩展	可选，未作正式结论	r39/r56 仅作开发补充

参考文献

- [1] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, et al. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. ICLR, 2025. <https://arxiv.org/abs/2405.14573>
- [2] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. AppAgent: Multimodal Agents as Smartphone Users. arXiv:2312.13771, 2023. <https://arxiv.org/abs/2312.13771>
- [3] Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. Mobile-Agent-E: Self-Evolving Mobile Assistant for Complex Tasks. arXiv:2501.11733, 2025. <https://arxiv.org/abs/2501.11733>
- [4] Ziwei Wang, Leyang Yang, Xiaoxuan Tang, Sheng Zhou, Dajun Chen, Wei Jiang, and Yong Li. History-Aware Reasoning for GUI Agents. arXiv:2511.09127, 2025. <https://arxiv.org/abs/2511.09127>
- [5] Weizhi Chen, Ziwei Wang, Leyang Yang, Sheng Zhou, Xiaoxuan Tang, Jiajun Bu, Yong Li, and Wei Jiang. PG-Agent: An Agent Powered by Page Graph. Proceedings of ACM Multimedia, 2025. <https://doi.org/10.1145/3746027.3755189>
- [6] Linqiang Guo, Wei Liu, Yi Wen Heng, Tse-Hsun Peter Chen, and Yang Wang. Agent-SAMA: State-Aware Mobile Assistant. Proceedings of the AAAI Conference on Artificial Intelligence, 40(35):29459–29467, 2026. <https://doi.org/10.1609/aaai.v40i35.40187>
- [7] Guangyi Liu, Pengxiang Zhao, Yaozhen Liang, et al. MemGUI-Bench: Benchmarking Memory of Mobile GUI Agents in Dynamic Environments. arXiv:2602.06075, 2026. <https://arxiv.org/abs/2602.06075>
- [8] Qiushi Sun, Mukai Li, Zhoumianze Liu, et al. OS-Sentinel: Towards Safety-Enhanced Mobile GUI Agents via Hybrid Validation in Realistic Workflows. Proceedings of ACL, pages 9529–9553, 2026. <https://aclanthology.org/2026.acl-long.431/>
- [9] Xue Yu, Bo Yuan, Pengshuai Yang, Kailin Zhao, Hong Hu, and Junlan Feng. SeerGuard: A Safety Framework for Mobile GUI Agents via World Model Prediction. arXiv:2607.15550, 2026. <https://arxiv.org/abs/2607.15550>
- [10] Kanzhi Cheng, Qiushi Sun, Yougang Chu, et al. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. Proceedings of ACL, pages 9313–9332, 2024. <https://aclanthology.org/2024.acl-long.505/>
- [11] Guijia Zhang and Harry Yang. Do GUI Agents Believe Their Eyes? Diagnosing State-Belief Reliance on Pixels versus Structure. arXiv:2607.04334, 2026. <https://arxiv.org/abs/2607.04334>

- [12] Leyang Yang, Ziwei Wang, Xiaoxuan Tang, et al. ProBench: Process-Aware Evaluation for GUI Agents. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(32):27547–27555, 2026. <https://ojs.aaai.org/index.php/AAAI/article/view/39974>
- [13] Qijun Han, Haoqin Tu, Zijun Wang, et al. VLAA-GUI: Knowing When to Stop, Recover, and Search, A Modular Framework for GUI Automation. *arXiv:2604.21375*, 2026. <https://arxiv.org/abs/2604.21375>
- [14] Yuzhe Zhang, Xianwei Xue, Xingyong Wu, et al. Don’t Act Blindly: Robust GUI Automation via Action-Effect Verification and Self-Correction. *Proceedings of ACL*, pages 28924–28941, 2026. <https://aclanthology.org/2026.acl-long.1335/>
- [15] Qinzhuo Wu, Pengzhi Gao, Wei Liu, and Jian Luan. BacktrackAgent: Enhancing GUI Agent with Error Detection and Backtracking Mechanism. *Proceedings of EMNLP*, pages 4250–4272, 2025. <https://aclanthology.org/2025.emnlp-main.212/>